

CLOSING THE SIM-TO-REAL GAP

A Thesis presented to
the Faculty of the Graduate School
at the University of Missouri

In Partial Fulfillment
of the Requirements for the Degree
Master of Science

by

LOGAN BRENNINGMEYER

Dr. Derek Anderson, Thesis Supervisor

May 2026

The undersigned, appointed by the Dean of the Graduate School, have examined the thesis entitled:

CLOSING THE SIM-TO-REAL GAP

presented by Logan Brenningmeyer,
a candidate for the degree of Master of Science and hereby certify that, in their
opinion, it is worthy of acceptance.

Dr. Derek Anderson

Dr. Jordan Malof

Dr. Matt Maschmann

Acknowledgments

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Table of Contents

Acknowledgments	i
List of Figures	v
List of Tables	vii
Abstract	viii
1 Introduction	1
2 Background	3
2.1 Sim-to-Real Gap	3
2.2 Levels of Domain Shift	3
2.3 Challenges	4
3 Related Work	6
3.1 Domain Adaptation	6
3.2 Augmentation-based Approaches	7
3.3 Image-to-Image Translation	7
3.3.1 GAN-based Translation	8
3.3.2 Diffusion-based Translation	8
4 Augmentation Evolution	10
4.1 Introduction	10
4.2 Background	11
4.2.1 Image Augmentations	11

4.2.2	Genetic Algorithms	11
4.3	Methods	13
4.3.1	Chromosome Design	13
4.3.2	Augmentation Set	14
4.3.3	Initialization	15
4.3.4	Fitness Evaluation	16
4.3.5	Genetic Operators	21
4.4	Experiments	22
4.4.1	Evolutionary Process	22
4.4.2	Object Classification Performance	26
4.5	Conclusion and Future Work	31
5	Pixel-Space CycleNet	34
5.1	Introduction	34
5.2	Background	34
5.2.1	Diffusion Models	34
5.2.2	Diffusion Sampling	36
5.2.3	Conditional Diffusion Models	37
5.2.4	ControlNet	39
5.2.5	Classifier-Free Guidance	39
5.2.6	Latent Diffusion Models	39
5.3	Methods	40
5.3.1	Overview	40
5.3.2	CycleNet	41
5.3.3	Pixel-Space CycleNet	42
5.3.4	Training Objectives	44
5.3.5	Training Setup	48
5.3.6	Inference Procedure	51

5.3.7	Evaluation Metrics	53
5.3.8	Model Selection	55
5.4	Experiments	56
5.4.1	Overview	56
5.4.2	Translation Behavior	59
5.4.3	Multi-Objective Selection Analysis	59
5.4.4	Downstream-Task Evaluation	59
5.5	Conclusion and Future Work	59
6	Conclusion	61
	Bibliography	62

List of Figures

4.1	Illustration of the main concept. Simulated (SIM) imagery is refined to better resemble real-world imagery. The SIM and real data are combined to train a classifier that operates on real imagery during evaluation or runtime. Performance is assessed on both modified and unmodified SIM data.	11
4.2	Illustration of the evolutionary process. Chromosomes (i.e., augmentation pipelines) are refined over generations based on feature-based sim-to-real similarity measures.	14
4.3	Median population fitness values and interquartile ranges (IQR) by generation. Values are averaged over three independent normalized runs for both fitness functions, CLIP and Layer1.	23
4.4	Probability distributions of normalized population fitness values at the initial generation (Gen. 0), the first reproduced generation (Gen. 1), and the final generation (Gen. 10). Values are averaged over three independent runs for both fitness functions: CLIP and Layer1.	24
4.5	Real, Simulated, CLIP, and Layer1 images for different samples.	26
4.6	Validation accuracy and F1 score of all classifier training runs for Base, CLIP, and Layer1.	28
4.7	Object A: CLIP, Layer1, and Base classifier testing log-scale ROC curves for object A's testing samples.	30

4.8	Object B: CLIP, Layer1, and Base classifier testing log-scale ROC curves for object B's testing samples.	30
4.9	Object C: CLIP, Layer1, and Base classifier testing log-scale ROC curves for object C's testing samples.	32
5.1	Forward noising of simulated images over noise strengths.	52
5.2	Caption	57
5.3	Caption	58
5.4	Enter Caption	58
5.5	Mean LPIPS between translated images and their source simulated images across CFG weight and noise strength configurations.	59
5.6	FID between translated images and their source simulated images across CFG weight and noise strength configurations.	60

List of Tables

4.1	Augmentation Set ($\mathcal{A}$) defining each available gene augmentation (A_k), its associated parameters ($\phi_{kl} \in \phi_k$), and each of its parameter's allowed range ($[r_{kl}^{\min}, r_{kl}^{\max}]$).	15
4.2	CLIP and Layer1 fittest individual augmentation pipelines and associated parameters. Highlighted cells indicates that the augmentation in its row is included in the pipeline for the fitness function in its column.	25
5.1	Summary of real and simulated datasets used in this work.	49
5.2	U-Net and CycleNet training dataset splits.	49
5.3	U-Net backbone training configuration.	50
5.4	CycleNet training configuration.	51
5.5	Final configurations selected for evaluation.	56

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1 Introduction

As the state of the art of computer vision has progressed into an era of data-driven deep learning, it has become increasingly important to possess large-scale, realistic training datasets [1]. Unlike many classical algorithms that rely on human-engineered features, deep neural networks learn task-relevant features by observing many training samples and iteratively optimizing parameters via gradient descent. While deep learning methods have proven to achieve state-of-the-art performance in a wide variety of domains [2], there is no guarantee that the learned features that emerge during training will generalize well to novel data. Therefore, it is important to supply deep learning models with a large quantity of diverse training data whose underlying features comprehensively and accurately reflect those of the unseen data.

When possible, it is ideal to train on data that was captured under the same conditions that will be used in deployment. For instance, when training an image classification model for real-world use onboard an aerial drone, the optimal approach is to train the model using images captured by the drone itself in the environments where it will be deployed. Training using deployment-relevant data provides the greatest assurance that the model will learn features that generalize to unseen data at inference time. Unfortunately, it can be impractical or infeasible to collect a substantial amount of real, diverse data for many real-world applications. As a result, training datasets are often supplemented using synthetically generated data, which can be produced at scale with high precision and significantly lower cost.

Advances in simulation tools such as video game engines [3] have made the pro-

duction of perceptually realistic images increasingly accessible, and it has been shown that an abundance of diverse simulated data can improve real-world performance [4]. However, simulated images inherently differ from real-world images in their underlying statistical properties, such as texture, noise, and frequency components. Deep learning computer vision models, which learn a hierarchy of feature representations at multiple scales, are naturally sensitive to these low-level characteristics. In addition, the training optimization process encourages models to exploit the most salient statistical patterns in the *training* data, regardless of their relevance at deployment. Consequently, differences in low-level features between simulated and real images can result in models learning representations that do not transfer effectively to real-world data. This discrepancy between simulated and real data is known as the *sim-to-real gap*, and it poses a significant challenge for training robust models in domains with limited real-world data.

Prior work has explored many approaches to bridging the sim-to-real gap. Domain adaptation methods aim to learn domain-invariant feature representations between simulated and real data, while augmentation-based approaches aim to improve generalization by increasing the diversity of simulated training data. More recently, image-to-image translation methods have been used to transform simulated images to better reflect their real-world counterparts. While these approaches have shown promising results, each presents tradeoffs in their expressiveness, training dynamics, and ability to preserve task-relevant features.

In this thesis, we investigate transformation-based approaches for bridging the sim-to-real gap. Through both image augmentations and generative image models, with a particular emphasis on diffusion-based methods, we transform simulated images to enhance their realism relative to unpaired real images to improve downstream image classification performance.

2 Background

2.1 Sim-to-Real Gap

In sim-to-real learning for computer vision tasks, models are trained on a source dataset of simulated images but deployed on a target dataset of real-world images. A model’s ability to generalize to real-world data depends on its ability to learn task-relevant features from simulated training data that are invariant to differences between the simulated and real domains. Let $p_s(x, y)$ and $p_t(x, y)$ denote the joint distributions over input-label pairs in the source (simulated) and target (real) domains, respectively, where x is the input image and y is the associated label for the downstream task. In practice, there are fundamental differences in the statistical properties of simulated and real data, i.e., $p_s(x, y) \neq p_t(x, y)$, which gives rise to the sim-to-real gap.

2.2 Levels of Domain Shift

The domain shift between simulated and real images can be generally organized into differences in their low-level statistics, mid-level structure, and high-level semantic information.

Low-level domain shift refers to differences in the statistical properties of pixel-level details, such as textures, noise, blur, color distributions, and frequency content. For instance, a simulated image rendered in a video game engine will not contain the same sensor noise as a real image captured by a drone. While sometimes imperceptible to the eye, these discrepancies can have significant effects on downstream models, as

early layers within deep neural networks are known to exploit low-level features such as local textures and high-frequency details. Therefore, models trained on simulated images whose low-level characteristics differ from the real target images may learn features that do not generalize to real-world data.

Mid-level domain shift refers to differences in the organization and structure of content between simulated and real images, such as object geometry, camera pose, and scene layout, often occurring due to simplifications or inaccuracies in the simulation process. For example, 3D models of real-world objects may have different geometries, sizes, poses, or scene placements than those typical of their real counterpart. Models trained on simulated data may struggle to generalize when their learned feature representations of spatial patterns do not match those seen in real-world imagery.

High-level domain shift arises from differences in semantic content between simulated and real data. This includes differences in class distributions or unrealistic scene compositions (e.g., a tree above the ground). Such high-level semantic differences may lead to models learning unrealistic relationships between input images and their labels, i.e., $p_s(y \mid x) \neq p_t(y \mid x)$, developing inaccurate representations of the contextual meaning and distribution of semantic content in real-world data.

2.3 Challenges

There are multiple factors concerning the practicality and complexity of closing the sim-to-real gap that make the problem nontrivial. For many vision tasks, simulated and real data are unpaired, meaning there is not a one-to-one correspondence between simulated and real images for supervised training. In such unpaired settings, image-to-image translation models cannot learn direct mappings between simulated images and their real counterparts. Instead, models must learn to either broadly apply a fixed transformation for all simulated images or adaptively apply a transformation conditioned by a given simulated image.

In addition, the sim-to-real gap arises at multiple scales, such as the low, mid, and high-level domain shifts described previously. Reconciling differences at one scale does not guarantee that others will align, and in fact, doing so can even widen their differences. For instance, an image-to-image translation model may shift an image’s color distribution closer to the real data distribution by similarly coloring grass and roads, destroying the high-level semantic content.

The objective of most sim-to-real learning is to improve a model’s ability to transfer to real data for some downstream task. Therefore, the true goal of sim-to-real image translation is not to improve perceptual realism but to transform the simulated data to enable models to learn task-relevant, discriminative features that generalize to real-world data. It is often the case that translated simulated data that is visually realistic either negligibly affects or even degrades downstream performance.

3 Related Work

The challenges of closing the sim-to-real gap have motivated a range of approaches, which are broadly organized into three main categories below: domain adaptation, augmentation-based methods, and image-to-image translation techniques.

3.1 Domain Adaptation

Domain adaptation methods aim to align feature representations across domains so that models trained on a source distribution, e.g., simulated images, perform well on a different target distribution, e.g., real images. While domain adaptation can be effective, it approaches the sim-to-real gap through alignment of feature representations, rather than directly addressing the differences in the input data.

A common approach is domain-adversarial training, which encourages models to learn discriminative yet domain-invariant features, that is, internal feature representations that improve performance while remaining indistinguishable across source and target domains. For instance, given features extracted by a neural network $f(x)$, a domain-adversarial neural network (DANN) [5] jointly trains a label classifier $C(f(x))$ and a domain classifier $D(f(x))$, encouraging learned features to improve label prediction while fooling the domain classifier.

Other domain adaptation approaches align feature representations by minimizing statistical differences across domains. Deep correlation alignment (CORAL) [6] minimizes the distance between the covariances of features, $\text{Cov}(f(x_s))$ and $\text{Cov}(f(x_t))$, across source and target domain inputs, respectively.

3.2 Augmentation-based Approaches

Augmentation-based approaches aim to improve generalization by increasing the variability of simulated training data through changes to scene construction, rendering parameters, or image properties. Though augmentations can improve generalization and are frequently used for many computer vision tasks, they are typically limited to predefined, parametric transformations. As a result, augmentations are often unable to capture the full breadth of differences between simulated and real-world characteristics.

Domain randomization [4], initially applied to robotic control, perturbs simulated images by augmenting properties such as camera pose, lighting, color, materials, etc., encouraging models to learn robust features without explicitly aligning their internal representations.

Learned augmentation-based approaches seek to optimize the selection of augmentations to improve task performance. AutoAugment [7] treats augmentations as a searchable space of augmentation pipelines and the parameters and strengths of their component augmentations. Searching through this space requires training many models, and therefore is very expensive. RandAugment [8] instead collapses the search space to the selection of N augmentations with a shared magnitude M , discarding the search space over the probabilities and parameters of individual augmentations.

3.3 Image-to-Image Translation

More recently, generative image models have been employed to directly translate simulated images to better reflect their real-world counterparts. Image-to-image translation models have shown encouraging results in perceptually bridging the sim-to-real gap, but they can be prone to introducing artifacts that may degrade performance in downstream models trained on the translated images.

3.3.1 GAN-based Translation

Early image-to-image translation methods employed generative adversarial networks (GANs) in both paired and unpaired settings. A foundational approach to paired image-to-image translation tasks (e.g., black-and-white to color) is pix2pix [9], which trains a conditional GAN to learn a mapping from conditioning source images x to output target images y . In unpaired settings, CycleGAN [10], jointly trains a pair of generative adversarial networks (GANs) to bidirectionally translate images between domains while enforcing *cycle consistency*—the property that the original images can be reconstructed following translation. Cycle consistency is a fundamental concept for many unpaired translation methods, as it encourages models to preserve structure without supervising training using paired output images.

3.3.2 Diffusion-based Translation

Diffusion models, capable of high-fidelity generation and wider mode coverage than GANs, have seen growing research interest for image-to-image translation. Training-free approaches such as SDEdit [11] use pre-trained diffusion models on the target domain to denoise corrupted inputs from the source domain. Training-free methods for prompt-conditional models may edit the style or content of an image generated by a source prompt (e.g., “cat on a bike”) by injecting its cross-attention maps into the denoising trajectory of a target prompt (e.g., “dog on a bike”) [12]. Other methods involve training diffusion models specifically for image-to-image translation tasks. For instance, Palette [13] extends SDEdit’s principles to pix2pix’s paired training framework, training diffusion models to denoise a corrupted target image while conditioned by its clean paired source image. CycleDiffusion [14] performs unpaired image-to-image translation by encoding a shared latent space between a pair of source-domain and target-domain diffusion models, constructing the latent code with the source-domain model while injecting noise then using the encoding to guide the denoising

trajectory of the target model. More recently, CycleNet [15] approaches image-to-image translation as a diffusion analogue of CycleGAN, injecting source images as conditioning and enforcing cycle consistency during training to preserve structure. CycleNet defines the source and target domains using prompt embeddings (e.g., “summer” and “winter”) and fine-tunes a pre-trained latent diffusion model to translate between them.

4 Augmentation Evolution

4.1 Introduction

In this chapter, we introduce a genetic algorithm (GA) designed to evolve an image augmentation pipeline for simulated images to close the sim-to-real gap for object classification datasets. Each individual in the population represents an ordered composition of non-repeating augmentations and their corresponding parameters. The initial population is formed by randomly sampling augmentations from a predefined, parameterized set constructed to reflect the properties of the real data.

An individual’s fitness is defined as the similarity between structurally similar pairs of real images and simulated images augmented using the individual’s pipeline. We consider two feature-based similarity metrics as fitness functions: cosine similarity between the feature embeddings from a pre-trained CLIP model [16], and cosine similarity between the feature embeddings of a classifier trained exclusively on real images. A high-level overview of our methods is shown in Fig. 4.1.

This approach enables the selection of augmentation pipelines that better align simulated images with real-world characteristics without disturbing their structure or requiring the need to retrain a downstream classifier to evaluate fitness.

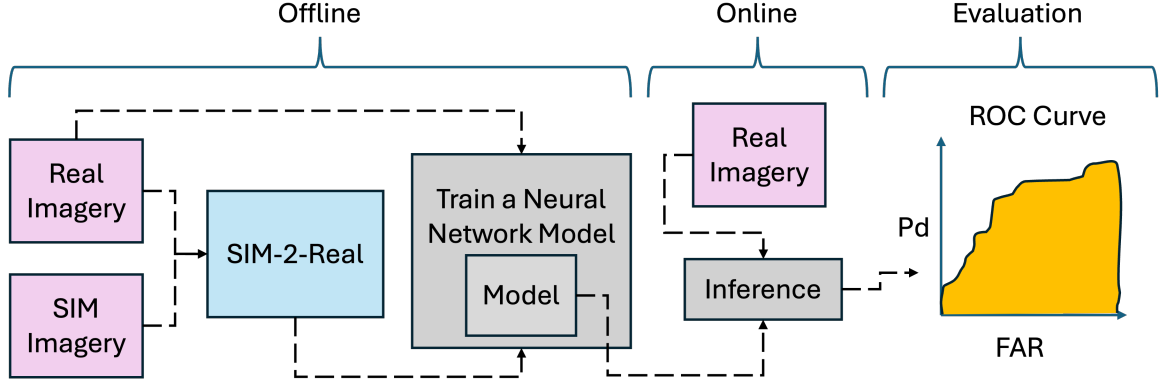


Figure 4.1: Illustration of the main concept. Simulated (SIM) imagery is refined to better resemble real-world imagery. The SIM and real data are combined to train a classifier that operates on real imagery during evaluation or runtime. Performance is assessed on both modified and unmodified SIM data.

4.2 Background

4.2.1 Image Augmentations

Image augmentations are fixed or parametric transformations applied to images, typically used to increase the diversity of training data to improve generalization. Augmentations can be broadly categorized into pixel-level transformations, such as color jitter, noise, blur, and contrast, and geometric transformations, such as rotation, translation, scaling, and flipping. Augmentations are typically applied randomly at some probability p with their parameters either fixed or randomly selected between defined lower and upper bounds (e.g., scale between 50-70% with probability $p = 0.5$).

4.2.2 Genetic Algorithms

Genetic Algorithms (GAs) are a class of evolutionary algorithms (EAs) inspired by the process of natural selection. A GA treats candidate solutions to a problem as individuals within a population, each represented as a chromosome composed of genes that encodes a particular solution. Over multiple generations, individuals reproduce and evolve based on their fitness, a heuristic representing how well an individual's

genes solve the problem at hand.

Chromosome Encoding

Chromosomes within a population may be encoded in a variety of ways depending on the task at hand. For instance, chromosomes may be encoded as binary strings where genes represent a sequence of bits. Genes may be encoded as digits, strings, functions, or any other data type or structure that is relevant to the task. In addition, chromosomes may contain a fixed or variable number of genes, and the genes may be ordered or unordered.

Fitness Function

The fitness function provides a real-valued evaluation of the quality, or fitness, of a solution, i.e., given a particular chromosome, how well does it solve the problem at hand? The fitness function is the driving force behind the optimization process, and therefore must be carefully designed given the choice of chromosome encoding and overall objective.

Selection

Selection determines which individuals within the current population are used to reproduce offspring based on fitness. Common approaches include roulette wheel selection, in which individuals are selected with probability proportional to their fitness, and tournament selection, in which subsets of the population “compete” and the fittest individual is selected. In addition, the selection process may incorporate elitism, retaining a subset of the fittest individuals in the next generation to ensure that the best solutions are not lost during crossover or mutation.

Crossover

The crossover operator defines how two parent solutions combine to create offspring for the next generation. Common approaches include single-point crossover, in which a single pair of genes is swapped between parents, two-point crossover, in which both parents' chromosomes are split into two sections and swapped, and uniform crossover, in which genes are mixed randomly.

Mutation

The mutation operator introduces random changes to individuals in the next generation by altering genes, allowing the population to explore new solutions. Depending on the chromosome encoding, mutation may entirely replace genes (e.g., flipping binary genes) or slightly perturb their values.

4.3 Methods

4.3.1 Chromosome Design

The GA ultimately produces augmentation pipelines: compositions of one or more parameterized augmentations applied in a distinct order. Over generations $g \in \{1, \dots, G\}$, a fitness function $f(\mathbf{x})$ guides the model to produce pipelines that enhance the realism of a simulated dataset $\mathcal{D}_{\text{train}}^{\text{sim}}$ in reference to a real dataset $\mathcal{D}_{\text{train}}^{\text{real}}$. To represent this process, the i^{th} individual of a population $\mathbf{x}_i^{(g)} \in \mathcal{X}^{(g)}$ is encoded as a chromosome (i.e., a vector of genes). The j^{th} gene of an individual $x_{ij} \in \mathbf{x}_i^{(g)}$ represents a specific configuration of an augmentation x_{ij}^A and its associated parameter values x_{ij}^ϕ . Individuals encoded in this way may then be used to create the augmentation pipelines formed by their genes as illustrated in Fig. 4.2

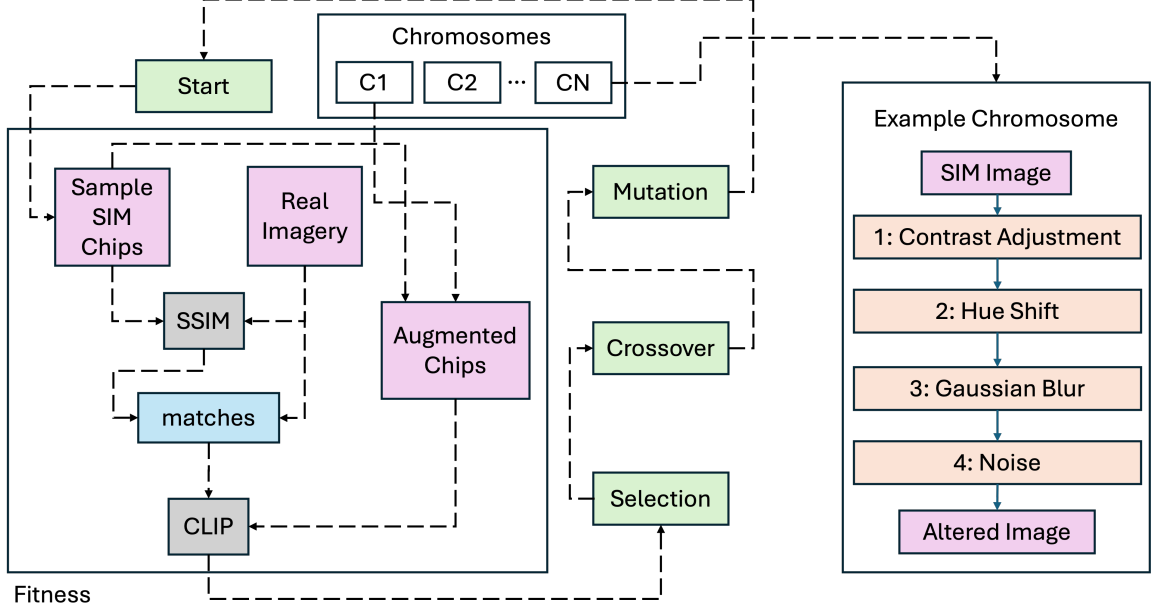


Figure 4.2: Illustration of the evolutionary process. Chromosomes (i.e., augmentation pipelines) are refined over generations based on feature-based sim-to-real similarity measures.

4.3.2 Augmentation Set

All genes are sampled from a predefined set of augmentations $\mathcal{A}$ containing K augmentation functions $A_k \in \mathcal{A}$ alongside their permitted parameter ranges $\mathbf{r}_k \in \mathcal{A}$:

$$\mathcal{A} = ((A_k, \mathbf{r}_k) \mid k = 1, \dots, K), \quad (4.1)$$

$$\text{where } \mathbf{r}_k = ([r_{kl}^{\min}, r_{kl}^{\max}] \mid l = 1, \dots, |\mathbf{r}_k|) \quad (4.2)$$

The index k of each augmentation $A_k \in \mathcal{A}$ defines its relative order within chromosomes, establishing the general order in which augmentations are ultimately applied. The dataset used in our experiments consists of aerial images of small objects; therefore, we selected and ordered augmentations to reflect common properties of real aerial imagery, such as compression, blur, and noise. However, the augmentation set can be adjusted to align with new datasets based on the properties of the real data. Table 4.1 lists the set of augmentations and parameter ranges we defined for

Table 4.1: Augmentation Set ($\mathcal{A}$) defining each available gene augmentation (A_k), its associated parameters ($\phi_{kl} \in \phi_k$), and each of its parameter’s allowed range ($[r_{kl}^{\min}, r_{kl}^{\max}]$).

Order	Augmentation	Parameters	Parameter Range
1	RandomBrightnessContrast	brightness_limit contrast_limit	$[-0.2, 0.2]$ $[-0.2, 0.2]$
2	HueSaturationValue	hue_shift_limit sat_shift_limit val_shift_limit	$[-5, 5]$ $[-10, 10]$ $[-10, 10]$
3	Downscale	scale_range	$[0.1, 0.9]$
4	GaussianBlur	blur_limit sigma_limit	$[3, 31]$ $[3, 31]$
5	MotionBlur	blur_limit	$[3, 31]$
6	MedianBlur	blur_limit	$[3, 31]$
7	BayerRG	None	None
8	GaussNoise	var_limit	$[0, 100]$
9	ISONoise	color_shift intensity	$[0, 0.2]$ $[0, 0.2]$

our experiments.

4.3.3 Initialization

At initialization, the GA spawns N_{pop} individuals as randomized, variable-length chromosomes (i.e., augmentation pipelines). Each individual in the initial population $\mathbf{x}_i^{(0)} \in \mathcal{X}^{(0)}$ is assigned a random chromosome length L_i , where $1 \leq L_i \leq K$. Each gene within a chromosome $x_{ij}^{(0)}$, $j \in \{1, \dots, L_i\}$ encodes a unique augmentation x_{ij}^A and its vector of parameter values x_{ij}^ϕ :

$$\begin{aligned}
 x_{ij} &= (x_{ij}^A, x_{ij}^\phi) \\
 \text{where } x_{ij}^A &= A_k \in \mathcal{A} \\
 x_{ij}^\phi &= (\phi_{kl} \in [r_{kl}^{\min}, r_{kl}^{\max}] \mid l = 1, \dots, |\mathbf{r}_k|)
 \end{aligned} \tag{4.3}$$

A gene’s augmentations are randomly sampled from $\mathcal{A}$ without replacement, ensuring no augmentations are reapplied. Each augmentation parameter ϕ_{kl} is initially assigned a normalized value $\phi_{kl}^{\text{norm}} \in [0, 1]$, sampled from a uniform distribution. This normalized value represents the relative position of the parameter’s value within its predefined bounds. The actual parameter value is then computed by denormalizing ϕ_{kl}^{norm} using its parameter bounds $[r_{kl}^{\min}, r_{kl}^{\max}]$:

$$\phi_{kl} = r_{kl}^{\min} + \phi_{kl}^{\text{norm}} \cdot (r_{kl}^{\max} - r_{kl}^{\min}) \quad (4.4)$$

4.3.4 Fitness Evaluation

The fitness function f serves to drive the GA to produce individuals whose augmentation pipelines narrow the sim-to-real gap. In other words, a greater fitness value indicates that an individual’s augmentation pipeline produces simulated images with greater realism. In an effort to capture the deep features underpinning the sim-to-real gap, we chose to leverage the rich feature embeddings of a pre-trained CLIP model as a means to compute similarity between real and simulated images. To ensure fitness is only computed using unbiased, informative comparisons between real and augmented simulated images, we devised a careful sampling strategy to select the images used to compute fitness.

Sampling Images for Comparison

As noted above, we define an individual’s fitness as a measure of its ability to realistically augment simulated images (i.e., its ability to make real and simulated images more similar). Therefore, we compute fitness by computing similarity measures of pairs of real and simulated samples. However, the quality and quantity of the images used for comparison is crucial for the effectiveness of the fitness value as an optimization signal. To ensure unbiased comparisons between high-quality image pairs, we

devised a sampling strategy consisting of three steps: i. Random Sampling, ii. Real Image Filtering, and iii. Context Matching.

i. Random Sampling We employ a mini-batch sampling approach, evaluating fitness on randomly selected subsets of the classifier dataset each generation. We first randomly sample S real images from $\mathcal{D}_{\text{train}}^{\text{real}}$ and S simulated images from $\mathcal{D}_{\text{train}}^{\text{sim}}$ for each class $c \in \{1, \dots, C\}$, sampling $2CS$ images total. As there is more than one class in our dataset, we chose to sample and compare images *per* class to ensure that similarity measures are not inadvertently affected by comparisons between different targets. Additionally, we only considered samples with tight target bounding boxes, maximizing the number of on-target pixels.

ii. Real Image Filtering Simulated images, by their nature, are consistently sharp and high-quality. On the other hand, real images are subject to quality-degradations such as blur, noise, and compression. To promote the likelihood that comparisons are made between high-quality images, we discard low-quality real samples using two levels of filtering. First, a blur filter removes real samples below some blurriness threshold. Second, a general quality filter removes real samples below a quality threshold of a fuzzy image quality metric. If any samples are filtered out, we repeatedly resample and filter replacements until we collect S high-quality real samples per class.

ii.i. Blur Filter For all CS sampled real images, $\mathcal{I}^{\text{real}} = \{I_{cs}^{\text{real}} \mid c = 1, \dots, C, s = 1, \dots, S\}$, blurriness is estimated using each image’s grayscale Laplacian variance $\sigma_{\nabla^2}^2(I_{cs}^{\text{real}})$. Using the Rec. 601 standard:

$$\tilde{I}_{cs}^{\text{real}} = 0.299 \cdot I_{cs}^{\text{real}}(R) + 0.587 \cdot I_{cs}^{\text{real}}(G) + 0.114 \cdot I_{cs}^{\text{real}}(B), \quad (4.5)$$

we convert each real sample I_{cs}^{real} to a grayscale image $\tilde{I}_{cs}^{\text{real}}$. Next, we apply the Laplacian operator to each grayscale image to obtain second-order information on the curvature of the image’s intensity gradient. This operation captures sharp changes in intensity, highlighting features such as corners and edges that contribute to image sharpness. The Laplacian of a grayscale image,

$$\nabla^2(\tilde{I}_{cs}^{\text{real}}) = \frac{\partial^2 \tilde{I}_{cs}^{\text{real}}}{\partial x^2} + \frac{\partial^2 \tilde{I}_{cs}^{\text{real}}}{\partial y^2}, \quad (4.6)$$

is computed as the sum of its second partial derivatives in x and y . We finally compute the variance of the Laplacian to quantify the spread of Laplacian values throughout the grayscale image. A low Laplacian variance indicates a lack of high-frequency content (i.e., the image is smooth and blurry). To ensure that only sufficiently sharp images are used for comparison, any real image whose Laplacian variance falls below a threshold t_{blur} is discarded, leaving the filtered set of real samples $\mathcal{I}^{\text{real}'}$. Formally, $\mathcal{I}^{\text{real}'}$ is defined as,

$$\mathcal{I}^{\text{real}'} = \{I_{cs}^{\text{real}} \in \mathcal{I}^{\text{real}} \mid \sigma_{\nabla^2}^2(\tilde{I}_{cs}^{\text{real}}) \geq t_{\text{blur}}\}, \quad (4.7)$$

where $\sigma_{\nabla^2}^2(\tilde{I}_{cs}^{\text{real}})$ is the Laplacian variance of the grayscale image.

ii.ii. Fuzzy Filter Following the blur filter, the remaining real images, $I_{cs}^{\text{real}} \in \mathcal{I}^{\text{real}'}$, are filtered further using a fuzzy logic system as a measure of image quality[17]. The fuzzy logic system first computes the values of several indicators of image quality, e.g., contrast, pixels on target, Haar wavelet energy focus, among others. These values are then aggregated into a single quality measure $Q(I)$ based on a manually defined fuzzy rule base. Similar to the blur filter, images whose quality measures are below

a threshold t_{fuzzy} are discarded, leaving the fully filtered set of real samples $\mathcal{I}^{\text{real}''}$:

$$\mathcal{I}^{\text{real}''} = \{I_{cs}^{\text{real}} \in \mathcal{I}^{\text{real}'} \mid Q(I_{cs}^{\text{real}}) \geq t_{\text{fuzzy}}\} \quad (4.8)$$

ii.iii. Replacement If any real samples are discarded by the filters, then for each class c where $|\mathcal{I}_c^{\text{real}''}| < S$ (i.e., one or more real samples were discarded from class c) we sample replacement images for the $S - |\mathcal{I}_c^{\text{real}''}|$ missing samples. We then repeatedly filter and resample images until we collect S high-quality real samples for all C classes, obtaining the final sets of real and simulated samples for comparison: $\mathcal{I}^{\text{real}*}$ and $\mathcal{I}^{\text{sim}*}$.

iii. Context Matching To prepare for the final fitness computation, we must construct a set of real/simulated image pairs to be used for comparison. Having obtained S simulated samples and S filtered real samples for all C classes, the simplest approach would be to randomly pair all real and simulated samples for each class for a total of CS image pairs. However, one of our concerns with comparing between arbitrarily paired images is that similarity measures can be inadvertently affected by “contextual differences”: differences in orientation, scale, time-of-day, etc. Using an augmentation set of standard, low-level image transforms, these contextual differences cannot be remedied by any augmentation pipeline produced by our GA. Therefore, selecting image pairs with matching context is crucial to ensure that fitness values capture similarity differences in terms of realism alone. We opted to use the structural similarity index measure (SSIM)[18] to compute the contextual similarity of image pairs. The SSIM of two images, I_1 and I_2 , is a weighted combination of three distinct comparison measures for luminance (l), contrast (c), and structure (s):

$$\text{SSIM}(I_1, I_2) = l(I_1, I_2)^\alpha \cdot c(I_1, I_2)^\beta \cdot s(I_1, I_2)^\gamma \quad (4.9)$$

SSIM’s joint evaluation of perceptual and spatial features makes it well-suited to act as a measure of contextual similarity. For each class, we compute an $S \times S$ SSIM similarity matrix, comparing all possible real/simulated image pairs between the sets of samples $\mathcal{I}^{\text{real}*}$ and $\mathcal{I}^{\text{sim}*}$. We then aggregate the top- M most similar pairs of each class, producing the set of image pairs, $\mathcal{P} = \{(P_{cm}^{\text{real}}, P_{cm}^{\text{sim}}) \mid c = 1, \dots, C, m = 1, \dots, M\}$, to be used for fitness evaluation.

Fitness Function

Every generation g , each individual’s fitness $f(\mathbf{x}_i^{(g)})$ is computed as a measure of its augmentation pipeline’s ability to improve the realism of the simulated images within $\mathcal{P}^{\text{sim}}$ in reference to $\mathcal{P}^{\text{real}}$.

CLIP Cosine Similarity To compute an individual’s fitness using CLIP feature embeddings, the individual’s augmentation pipeline is first applied to every simulated image in $\mathcal{P}^{\text{sim}}$, producing a set of pairs of real and augmented simulated images $\mathcal{P}^{\text{aug}} = \{(P_{cm}^{\text{real}}, P_{cm}^{\text{aug}}) \mid c = 1, \dots, C, m = 1, \dots, M\}$. For each image pair in $\mathcal{P}^{\text{aug}}$, both images are fed into CLIP to obtain 512-dimensional feature embeddings, $\text{CLIP}(P_{cm}^{\text{real}})$ and $\text{CLIP}(P_{cm}^{\text{aug}})$. Fitness $f_{\text{CLIP}}(\mathbf{x}_i^{(g)})$ is finally computed as the average cosine similarity of the CLIP feature embeddings for all pairs in $\mathcal{P}^{\text{aug}}$.

$$f_{\text{CLIP}}(\mathbf{x}_i^{(g)}) = \frac{1}{CM} \sum_{c=1}^C \sum_{m=1}^M \frac{\text{CLIP}(P_{cm}^{\text{real}}) \cdot \text{CLIP}(P_{cm}^{\text{aug}})}{\|\text{CLIP}(P_{cm}^{\text{real}})\| \cdot \|\text{CLIP}(P_{cm}^{\text{aug}})\|} \quad (4.10)$$

Layer1 Cosine Similarity Alternatively, to compute an individual’s fitness using feature embeddings intrinsically tied to our object classification task, we trained a ResNeXt classifier on exclusively real images from our dataset. We reasoned that if given only real images, the classifier would learn real-data-specific features, thereby allowing it to capture the realism of augmented images without the influence of learned features of simulated samples. As when computing $f_{\text{CLIP}}(\mathbf{x}_i^{(g)})$, we extract feature

embeddings for each image pair in $\mathcal{P}^{\text{aug}}$. Precisely, feature embeddings are taken from the first residual block, “Layer1”, of the classifier. Fitness $f_{\text{Layer1}}(\mathbf{x}_i^{(g)})$ is then computed as the average cosine similarity of the Layer1 feature embeddings for all pairs in $\mathcal{P}^{\text{aug}}$.

$$f_{\text{Layer1}}(\mathbf{x}_i^{(g)}) = \frac{1}{CM} \sum_{c=1}^C \sum_{m=1}^M \frac{\text{Layer1}(P_{cm}^{\text{real}}) \cdot \text{Layer1}(P_{cm}^{\text{aug}})}{\|\text{Layer1}(P_{cm}^{\text{real}})\| \cdot \|\text{Layer1}(P_{cm}^{\text{aug}})\|} \quad (4.11)$$

4.3.5 Genetic Operators

Having computed the fitness values for the full population, the GA then produces the next generation $\mathcal{X}^{(g+1)}$ using selection, crossover, and mutation operators.

Selection To prepare to reproduce the next generation, we first select the N_p fittest individuals as the parents used for reproduction, $\mathcal{X}_p^{(g)}$. Additionally, we take a fraction $p_e \in [0, 1)$ of the population as elite individuals, $\mathcal{X}_e^{(g)}$, to be passed to the next generation. The number of elites is defined as as:

$$N_e = \lfloor p_e N_{\text{pop}} \rfloor \quad (4.12)$$

Crossover Following selection, we produce new offspring by performing random gene crossover between random pairs of parents. For each of the $N_{\text{pop}} - N_e$ individuals left to be added to the next generation $\mathcal{X}^{(g+1)}$, we randomly select two parents $\mathbf{x}_{p_1}^{(g)}, \mathbf{x}_{p_2}^{(g)} \in \mathcal{X}_p^{(g)}$ for reproduction. To produce an offspring, the genes (i.e., augmentations) of both parents are first combined into a set of genes $\mathcal{G}_{p_1, p_2}$, excluding duplicate augmentation types:

$$\mathcal{G}_{p_1, p_2} = \{x \in \mathbf{x}_{p_1}^{(g)} \cup \mathbf{x}_{p_2}^{(g)} \mid x^A \text{ is unique}\} \quad (4.13)$$

Each offspring $\mathbf{x}_i^{(g+1)}$ is then assigned a random chromosome length $L_i \in \{1, \dots, \min(K, |\mathcal{G}_{p_1, p_2}|)\}$ with genes randomly sampled from $\mathcal{G}_{p_1, p_2}$.

Mutation To promote the exploration of new solutions, we mutate genes by assigning random values to randomly selected gene augmentation parameters throughout the population $\mathcal{X}^{(g+1)}$. For a given augmentation parameter ϕ_{kl} , values are sampled within its allowed range $[r_{kl}^{\min}, r_{kl}^{\max}]$ as defined in $\mathcal{A}$. Parameters mutate according to a mutation rate $p_m \in [0, 1]$ that defines a fixed probability of mutation for all parameters.

4.4 Experiments

4.4.1 Evolutionary Process

For both fitness functions, f_{CLIP} and f_{Layer1} , we performed three separate evolution runs with fixed parameters.

Evolution Parameters

Population All GA runs were conducted using $G = 10$ generations of $N_{\text{pop}} = 200$ individuals containing $N_p = 20$ parents in each. The proportion of individuals chosen as elites $p_e = 0.1$. The mutation rate $p_m = 0.1$ as well, meaning all augmentation parameters have a 10% chance of random reassignment each generation.

Sampling The sampling parameters used to select images for fitness evaluation were based on manual tests of multiple configurations and qualitative judgments of their sample quality. Each generation, $S = 25$ real and simulated images are randomly sampled from each class. All real samples are then passed through the blur and fuzzy filters using threshold values of $t_{\text{blur}} = 1.5$ and $t_{\text{fuzzy}} = 0.45$. For each class $c \in \{1, \dots, C\}$, the $M = 5$ unique sample pairs with the highest SSIM values are

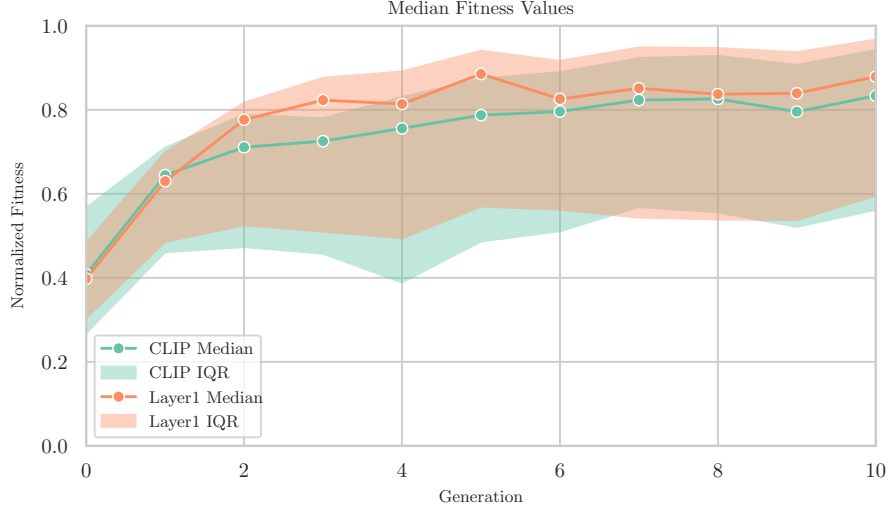


Figure 4.3: Median population fitness values and interquartile ranges (IQR) by generation. Values are averaged over three independent normalized runs for both fitness functions, CLIP and Layer1.

selected to be used for fitness evaluation.

An additional fixed set of $M \times C$ image pairs are sampled before the initial generation to balance random sample variability. Therefore, each individual’s fitness is evaluated using $2M \times C$ image pairs: $M \times C$ fixed pairs and $M \times C$ random pairs sampled each generation.

Fitness Functions Analysis

To validate the utility of our fitness functions as stable optimization signals, we examined generational trends in populations’ fitness values for each. Fig. 4.3 shows the median normalized fitness values and interquartile ranges over generations for both fitness functions. Specifically, the plotted values represent averages over the runs of each fitness function.

Both fitness functions demonstrate an initial stable upwards trend in population fitness, beginning to plateau around generation six. The consistent improvement in interquartile range indicates that learned augmentations have a reliably positive im-

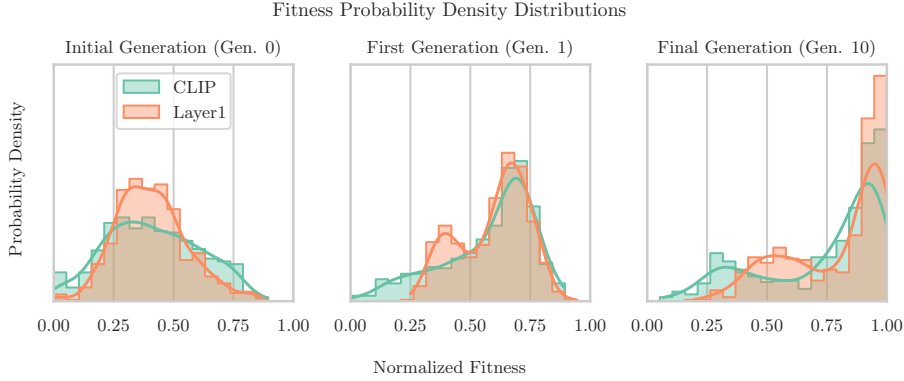


Figure 4.4: Probability distributions of normalized population fitness values at the initial generation (Gen. 0), the first reproduced generation (Gen. 1), and the final generation (Gen. 10). Values are averaged over three independent runs for both fitness functions: CLIP and Layer1.

pact on fitness. This is illustrated more clearly in Fig. 4.4, which displays the shift in fitness distributions over generations. Populations gravitate strongly towards higher fitness values in clusters, implying that the fitness functions are nuanced enough to assign similar fitness scores to genetically similar individuals.

Augmentation Pipelines

Table 4.2 displays the augmentation pipelines of the fittest individuals among all runs for each fitness function. As depicted by the highlighted cells, the fitness functions ultimately converged to use the same set of augmentations. Moreover, they seem to reach a general consensus on both the magnitude and sign of many of the augmentation parameters. For instance, both pipelines set the magnitudes of their blur augmentations in the same descending order: GaussianBlur, then MotionBlur, followed by MedianBlur. Additionally, with the sole exception of hue_shift_limit, both pipelines converge to the same parameter signs. The similarity between the pipelines suggests that CLIP embeddings and classifier features capture similar real-data characteristics.

Examples of augmented simulated images produced by the GA augmentation

pipelines are shown in Fig. 4.5. The first two columns of each row contain real and simulated image pairs matched using the fitness sampling strategy. All images were sampled from our false alarms class, meaning they do not contain any labeled objects and consist of only backgrounds and miscellaneous objects. The last two columns contain the augmented simulated images produced by the CLIP and Layer1 augmentation pipelines with the greatest fitness. The most significant visual difference between the augmented images is their differing levels of brightness and saturation. The CLIP pipeline’s images are both brighter and less saturated than Layer1’s due to CLIP’s higher RandomBrightnessContrast brightness_limit, HueSaturationValue val_shift_limit, and ISONoise color_shift values.

Table 4.2: CLIP and Layer1 fittest individual augmentation pipelines and associated parameters. Highlighted cells indicates that the augmentation in its row is included in the pipeline for the fitness function in its column.

Chromosome Order	Augmentation	Parameters	CLIP	Layer1
1	RandomBrightnessContrast	brightness_limit contrast_limit	0.169 -0.093	0.044 -0.188
2	HueSaturationValue	hue_shift_limit sat_shift_limit val_shift_limit	4.519 -4.511 -0.879	-0.629 -9.903 -4.001
3	Downscale	scale_range	0.621	0.353
4	GaussianBlur	blur_limit sigma_limit	31 27	31 29
5	MotionBlur	blur_limit	7	27
6	MedianBlur	blur_limit	5	13
7	BayerRG	None		
8	GaussNoise	var_limit		
9	ISONoise	color_shift intensity	0.099 0.151	0.043 0.02

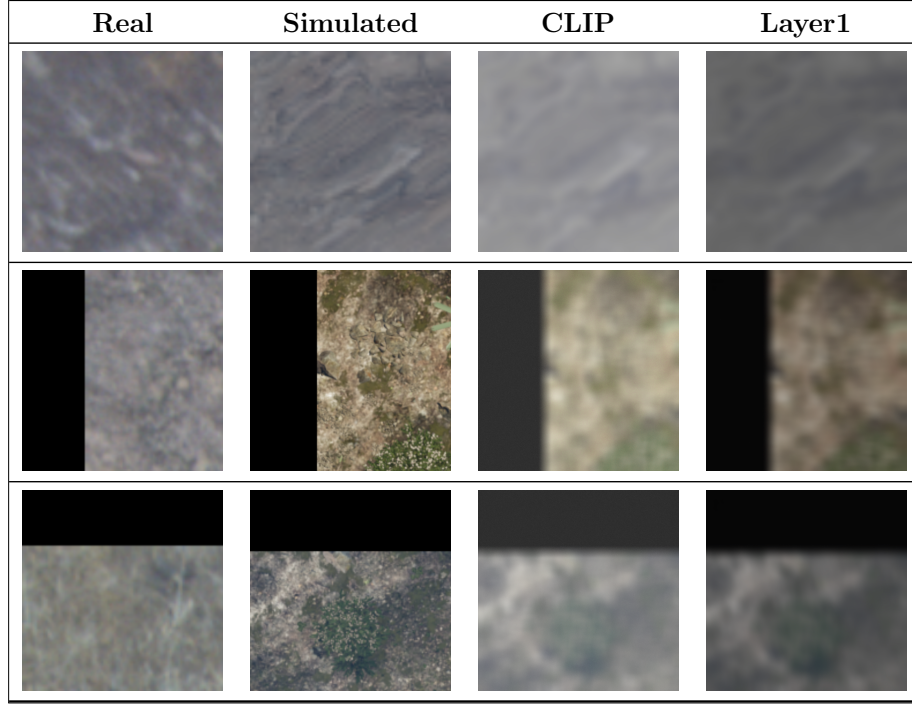


Figure 4.5: Real, Simulated, CLIP, and Layer1 images for different samples.

4.4.2 Object Classification Performance

To evaluate the effect of augmented simulated data on the downstream classification task, we trained three ResNeXt classifiers for both fitness functions, producing three “CLIP classifiers” and three “Layer1 classifiers”. Each fitness function’s classifiers were trained on a modified version of our original training dataset, where all simulated images were augmented using the corresponding fitness function’s augmentation pipeline. To set a performance baseline for comparison, we additionally trained three “Base classifiers” on the original training dataset. We then compared the performance of these classifiers using receiver operating characteristic (ROC) curves, with values averaged over their runs.

Datasets

The classifiers were trained using the same datasets, $\mathcal{D}_{\text{train}}^{\text{real}}$ and $\mathcal{D}_{\text{train}}^{\text{sim}}$, that were used to optimize the GA. The Base classifiers were trained using unmodified real and

simulated images, while the CLIP and Layer1 models were trained using unmodified real images and augmented simulated images, denoted $\mathcal{D}_{\text{train}}^{\text{aug}}$.

The validation dataset consisted of a randomly selected and withheld subset of $\mathcal{D}_{\text{train}}^{\text{real}} \cup \mathcal{D}_{\text{train}}^{\text{sim}}$, containing both real and simulated images. After class balancing the training dataset, the training and validation datasets yielded a 94 : 6 split, respectively. We additionally tested models on a withheld dataset of real images to evaluate how the models perform in a real-world environment.

The real and simulated datasets were generated as follows. The simulated dataset was created using Unreal Engine 5.3, while the real dataset consisted of aerial drone imagery. No explicit effort was made to ensure a high degree of similarity between the real and simulated datasets, such as constructing a digital twin; the only commonality was the use of similar biomes with different content. The primary objective was to determine whether the proposed methods could reduce the gap between simulated and real data, thereby improving the usefulness of simulated data for localization and detection of different objects. Future work will focus on applying these techniques to benchmark aerial datasets, though a key challenge lies in identifying relevant simulated aerial data that aligns with existing real-world datasets.

Classifier Training Setup

Each classifier was trained for 200 epochs on approximately 570,000 images after oversampling to balance classes. In addition, all classifiers were trained with a batch size of 128, AdamW[19] optimization, and a learning rate of 0.0001. To account for potential random variations in the training process, we performed three training runs per model type. The full min-max envelope between runs for each model type is indicated by the shaded regions in the ROC curve plots.

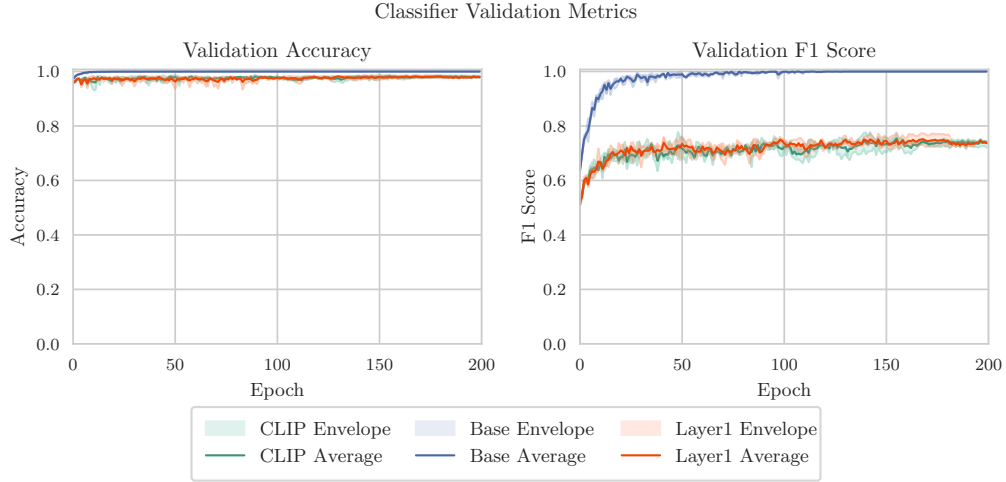


Figure 4.6: Validation accuracy and F1 score of all classifier training runs for Base, CLIP, and Layer1.

Validation Metrics

As stated previously, the validation dataset used for all classifier models contained a random mix of real and simulated images. We selected each classifier’s epoch used for testing as the training epoch that achieved the lowest validation error. However, as the simulated images within the validation dataset were not augmented, the CLIP and Layer1 classifiers struggled to achieve stable convergence on the validation dataset during training as shown in Fig 4.6. As a result, the testing epochs of the CLIP and Layer1 classifiers were all produced before the 100th training epoch. On the other hand, the testing epochs of the Base classifier were all produced near the final 200th training epoch.

Testing Results

In this section, we analyze each model type’s performance on the testing dataset using the receiver operating characteristic (ROC) curves of their runs. For each model type (i.e., CLIP, Layer1, and Base) we plot the log-scale ROC curves of the epoch with the lowest validation error for all three runs. Each object type in our dataset is represented

in different ratios between $\mathcal{D}_{\text{train}}^{\text{real}}$ and $\mathcal{D}_{\text{train}}^{\text{sim}}$; some objects possess mostly real images, others are mostly simulated. As such, and as stated above, we provide object-specific ROC curves for object types A, B, and C, arranged in ascending order of classification difficulty on our testing dataset as demonstrated by previous experiments. To provide a fair comparison between classifiers’ performance, we provide measures for area under the curve (AUC) and average true positive rate ($\overline{\text{TPR}}$) at a fixed false positive rate (FPR) threshold T .

Of object A’s $\sim 9,000$ total training samples, 72.86% come from real data. Due to its large quantity of real data, our classifiers consistently perform well on object A when testing on real images. As shown in Fig. 4.7, all classifiers achieve similar $\overline{\text{TPR}}$ past the initial high confidence thresholds. However, the Base classifier achieves a $\overline{\text{TPR}}$ of ~ 0.8 at the highest confidence threshold, much higher than the CLIP and Layer1 classifiers’ $\overline{\text{TPR}}$ values of ~ 0.4 and ~ 0.2 , respectively. The Base classifier’s high $\overline{\text{TPR}}$ indicates that it can classify a subset of nearly 80% of object A’s testing samples with extremely high confidence. On the other hand, the CLIP and Layer1 classifiers exhibit a more gradual increase in $\overline{\text{TPR}}$ at the highest confidence thresholds; their confidence values are more widely spread among testing samples. The models’ differing levels of performance at high confidence thresholds can likely be attributed to differences in the epochs used for testing; the CLIP and Layer1 classifiers’ runs achieved minimum validation error at much earlier epochs than the Base classifier’s runs. As a result, their tested epochs did not experience enough training iterations to produce the extremely confident predictions of the Base classifier.

Object B consists of only 3.89% real images of its $\sim 13,000$ total training samples, with the remaining 96.11% being simulated. As a result, our classifiers can leverage learned features from the simulated data to obtain a slight performance improvement on real data. As shown in Fig. 4.8, all classifiers perform very similarly at the highest confidence threshold, with $\overline{\text{TPR}} \approx 0.1$ for each. As the confidence threshold increases,

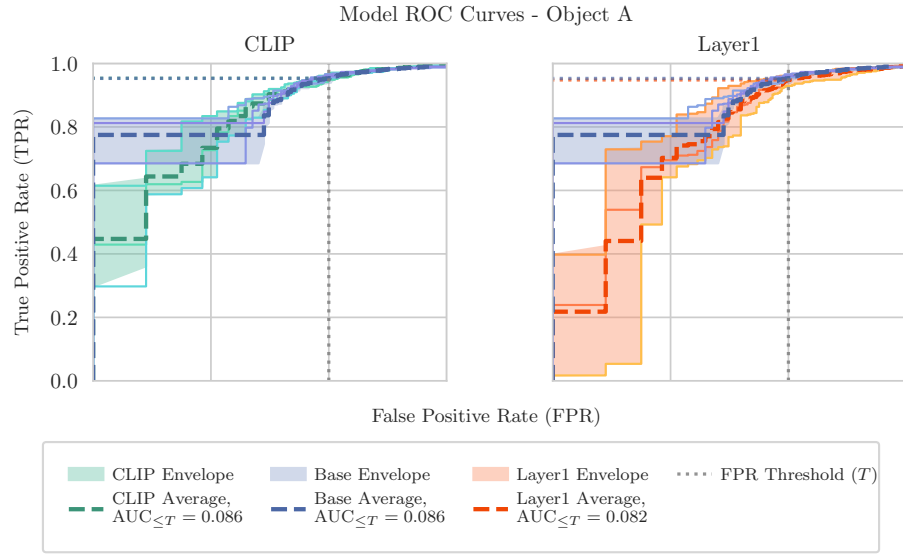


Figure 4.7: Object A: CLIP, Layer1, and Base classifier testing log-scale ROC curves for object A's testing samples.

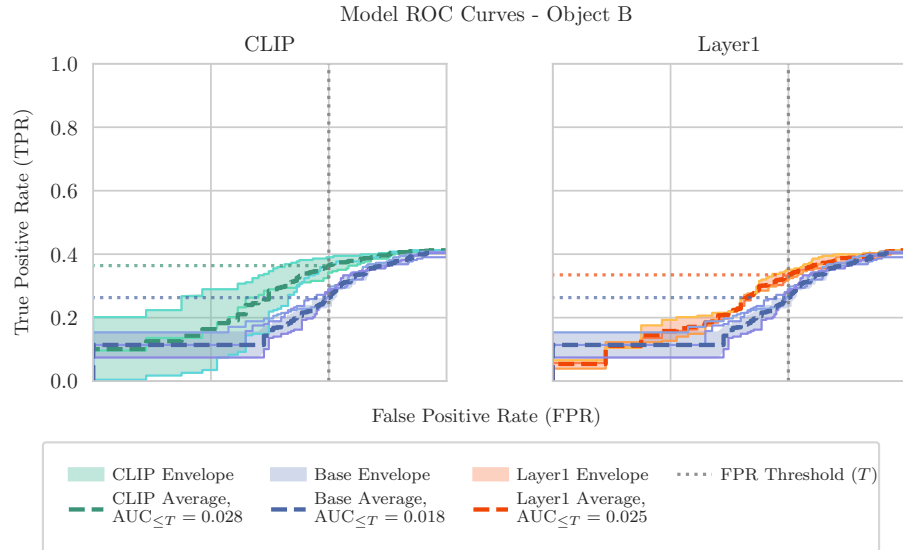


Figure 4.8: Object B: CLIP, Layer1, and Base classifier testing log-scale ROC curves for object B's testing samples.

the CLIP and Layer1 classifiers begin to attain higher $\overline{\text{TPR}}$ values than the Base classifier, with the CLIP model showing the greatest improvement. Specifically, at $\text{FPR} = T$, the CLIP classifier’s $\overline{\text{TPR}} = 0.364$ and the Layer1 classifier’s $\overline{\text{TPR}} = 0.335$, while the Base classifier’s $\overline{\text{TPR}} = 0.263$. Moreover, the CLIP and Layer1 classifiers’ $\text{AUC}_{\leq T}$ values of 0.028 and 0.025, respectively, surpass the Base classifier’s at 0.018. These performance improvements suggest that object B’s high proportion of simulated images renders it particularly well-suited to receive performance improvements from training on augmented simulated images.

Object C is one of the least well-represented objects in our training dataset, containing only $\sim 2,000$ samples total, making it particularly difficult to classify in unseen data. However, it has the most even balance of real and simulated data among the objects in the testing dataset, containing 65.59% real samples and 34.41% simulated samples. In Fig. 4.9, the CLIP and Layer1 classifiers achieve $\overline{\text{TPR}}$ values of 0.284 and 0.272, respectively, where $\text{FPR} = T$. On the other hand, the Base classifier attains a slightly lower $\overline{\text{TPR}}$ of 0.23. Likewise, the $\text{AUC}_{\leq T}$ values of CLIP and Layer1 see a slight increase from the Base classifier’s value of 0.016 to 0.021 and 0.02, respectively. Though object C contains a much lower quantity of simulated images compared to object B, we reason that its general scarcity of samples and reasonably balanced dataset provide the augmented simulated images with enough influence to return an improvement in performance.

4.5 Conclusion and Future Work

In this paper, we presented a preliminary evolutionary approach for closing the sim-to-real gap in object classification datasets. In our approach, GAs evolve to produce interpretable augmentation pipelines that improve the similarity between carefully selected pairs of real and simulated images. We compared the evolution behavior of GA runs configured with different fitness functions, i.e., measures of similarity be-

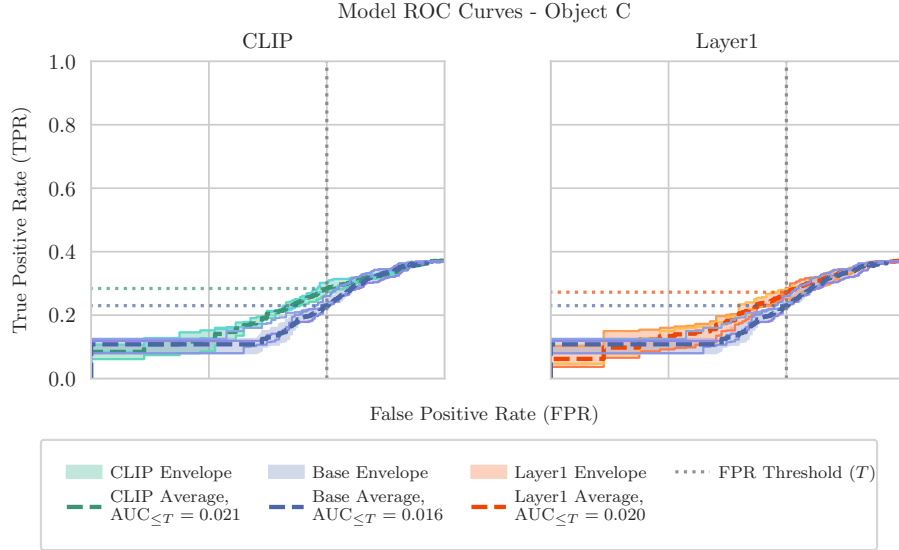


Figure 4.9: Object C: CLIP, Layer1, and Base classifier testing log-scale ROC curves for object C’s testing samples.

tween image pairs: one based on the feature similarity of a pre-trained CLIP model (CLIP) and the other on the feature similarity of an in-house classifier trained on real images (Layer1). To assess the impact of the GA pipelines on object classification performance, we first trained multiple classifier models on datasets containing augmented simulated images produced by the GA pipelines. We then compared a selection of their ROC curves with those of a baseline classifier trained on the original dataset. Our experimental results suggest that classifiers trained on the augmented simulated images achieve better performance on objects with an insufficient amount of real data. However, objects possessing a high-quantity of real data see no significant improvement nor degradation in performance.

In future work, we plan to investigate other image similarity measures or their combinations for use as fitness functions (e.g., Contrastive Anchor Network[20] feature similarity), additionally trained using different validation sets to evaluate how model selection impacts our test results. We also plan to conduct further experiments using alternative augmentation sets to assess the impact of varying augmentations and their parameter bounds. Moreover, we are looking into ways to prevent the produc-

tion of duplicate or functionally equivalent individuals, thereby promoting population diversity and the exploration of novel solutions. Last, we believe our current approach is focused on deriving transformations that make the machine features more similar. While this is important, it is also of interest to explore alterations on our process such that learned transformations also “look”, according to a human, perceptually closer.

5 Pixel-Space CycleNet

5.1 Introduction

In this chapter, we introduce Pixel CycleNet, an adaptation of the CycleNet [15] architecture from the latent space to pixel space.

5.2 Background

5.2.1 Diffusion Models

Diffusion models are a class of generative models that generate samples via an iterative denoising process in which a neural network (typically a U-Net [21]) predicts the noise injected into clean images sampled from a data distribution. Once trained, they generate samples by gradually refining pure noise over many steps of noise prediction and removal.

The diffusion process consists of a forward process that corrupts clean images and a reverse process that denoises the corrupted images using the model’s noise predictions. Both processes follow a noise schedule β_t that defines the variance of the noise added at timestep $t \in \{1, \dots, T\}$, where the forward process progresses from 1 to T , while the reverse process descends from T to 1.

Forward Process

Given a clean image $x_0 \sim p(x)$, the forward process iteratively corrupts the image with small amounts of Gaussian noise at each timestep, producing a sequence of

increasingly corrupted images $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$, until x_T is approximately pure Gaussian noise: $x_T \sim \mathcal{N}(0, I)$. Each successive corrupted image x_t is produced by injecting Gaussian noise with variance β_t into the preceding image x_{t-1} . This forward process can be expressed as a sequence of Gaussian transitions $q(x_t | x_{t-1})$.

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I)$$

$$\alpha_t = 1 - \beta_t.$$

In practice, the forward process can be condensed into a closed-form equation for any timestep t , where x_t is obtained from x_0 directly without progressing through the intermediate steps:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{5.1}$$

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Rearranging the closed form allows us to estimate the original clean image x_0 from a noisy sample x_t and a noise prediction $\epsilon_\theta(x_t, t)$:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{5.2}$$

Reverse Process

Beginning from pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$, the reverse process iteratively removes noise to recover a clean image as t descends from T to 1. Each step of the reverse process is modeled as a conditional distribution $p_\theta(x_{t-1} | x_t)$, parameterized by a neural network that predicts the noise present in x_t . The model's noise predictions are used to produce progressively less noisy samples at each timestep until a clean sample x_0 is obtained.

Training

Typically, diffusion models are trained to predict the noise ϵ_t that has been injected into a noisy sample x_t . Given a batch of clean samples $\{x_0^{(i)}\}_{i=1}^B$, we randomly sample timesteps $t \sim \mathcal{U}(\{1, \dots, T\})$ and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$, and construct a batch of noisy samples $\{x_t^{(i)}\}_{i=1}^B$ using Eq. (5.1). The model then predicts the noise $\epsilon_\theta(x_t, t)$ given the noisy sample and its corresponding timestep. The model is trained to minimize the mean squared error (MSE) between its predicted noise and the true noise used to construct x_t :

$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \|\epsilon_\theta(x_t, t) - \epsilon\|_2^2.$$

5.2.2 Diffusion Sampling

There are two core procedures for parameterizing $p_\theta(x_{t-1} \mid x_t)$ to generate samples from pure Gaussian noise: the stochastic sampling procedure introduced in Denoising Diffusion Probabilistic Models (DDPM) [22] and the deterministic sampling procedure introduced in Denoising Diffusion Implicit Models (DDIM) [23].

DDPM

DDPM sampling parameterizes the reverse transition $p_\theta(x_{t-1} \mid x_t)$ as a Gaussian distribution with mean $\mu_\theta(x_t, t)$ and variance $\sigma_t^2 I$, where σ_t^2 is typically set to either β_t or the posterior variance $\tilde{\beta}_t$. A sample x_{t-1} is then obtained by adding Gaussian noise scaled by σ_t to the mean:

$$\begin{aligned} \mu_\theta(x_t, t) &= \frac{1}{\alpha_t} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \\ x_{t-1} &= \mu_\theta(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) \end{aligned}$$

DDIM

DDIM sampling defines the reverse process as a deterministic update from x_t to x_{t-1} , without requiring sampling noise at each timestep. Additionally, DDIM allows for the inclusion of a tunable level of stochastic noise as a special case. Given x_t and $\epsilon_\theta(x_t, t)$, DDIM computes an estimate of the clean image $\hat{x}_0$ using Eq. (5.2) and constructs x_{t-1} as a deterministic step towards $\hat{x}_0$, scaling the noise level accordingly for the previous timestep $t - 1$:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t).$$

Due to the consistent denoising trajectory that DDIM sampling provides, the number of denoising steps can be significantly reduced by skipping intermediate timesteps, significantly improving efficiency without degrading sample quality. Given its efficiency and deterministic behavior, DDIM is commonly used in practice, and is the sampling procedure adopted in our experiments.

5.2.3 Conditional Diffusion Models

A standard diffusion model is designed to generate samples unconditionally, meaning it learns the reverse transitions $p_\theta(x_{t-1} \mid x_t)$ using only the context provided by the noisy sample x_t . On the other hand, the denoising trajectory of a conditional diffusion model is influenced by one or more conditioning inputs c , e.g., an image or text prompt, and learns the conditional reverse transitions $p_\theta(x_{t-1} \mid x_t, c)$, where the noise is predicted by $\epsilon_\theta(x_t, t, c)$.

Image Conditioning

Image conditioning enables diffusion models to incorporate conditioning images c_{img} (e.g., depth maps, segmentation masks, Canny edges, etc.) into its noise predictions $\epsilon_\theta(x_t, t, c_{\text{img}})$. This allows generated samples to align with properties of the condition-

ing image, such as its spatial layout, edges, and semantic content. Image-conditioned diffusion models are often fundamental for image-to-image translation tasks, where preserving the structural and semantic information of source imagery is crucial.

In practice, conditioning images are encoded into feature representations which are then integrated into the denoising network. While conditioning images may simply be concatenated with the noisy sample x_t , they are more commonly encoded using auxiliary networks and combined with the base network’s intermediate feature representations, often through addition or concatenation.

Text Conditioning

Text conditioning enables diffusion models to incorporate a text prompt c_{text} into the noise prediction $\epsilon_{\theta}(x_t, t, c_{\text{text}})$, aligning the content of generated samples with the semantic content of the text. Text-conditioned diffusion models may also be used for image-to-image translation when the characteristics of the source and target domains can be precisely described through text. However, there is no guarantee that the encoded prompts accurately or fully capture the underlying properties of the data.

In modern architectures, text conditioning is commonly incorporated by first extracting prompt embeddings using a text encoder (e.g., CLIP [16]) and then injecting them into the denoising network at multiple layers using cross-attention. Text conditioning provides more expressive, abstract control over generation, allowing the inclusion of any context that can be meaningfully described using text. However, the effects of a prompt depend both on how precisely it specifies the desired content and on the semantic associations learned by the text encoder, which may include undesired concepts not intended by the prompt.

5.2.4 ControlNet

5.2.5 Classifier-Free Guidance

Conditional diffusion models may be too weakly influenced by the conditioning input c , potentially generating high-quality samples yet disregarding the condition’s spatial or semantic content. Classifier-free guidance (CFG) is a method of strengthening conditioning by interpolating the model’s noise prediction towards its conditional prediction $\epsilon_\theta(x_t, t, c)$ and away from an unconditional prediction $\epsilon_\theta(x_t, t, \emptyset)$ where the condition is dropped, e.g., an empty prompt string or zero tensor image.

During training, the model is randomly given a conditioning input c with some probability, with the condition dropped otherwise. When generating a sample, the model performs both a conditional pass incorporating the condition and an unconditional pass with the condition dropped. The model’s final noise prediction $\tilde{\epsilon}_\theta(x_t, t, c)$ is then computed as a linear interpolation between the unconditional and conditional predictions. The strength of the pull towards the conditional prediction controlled by a guidance scale w , where a guidance scale of 1 is equivalent to performing a standard conditional pass:

$$\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t, \emptyset) + w(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)). \quad (5.3)$$

Increasing the guidance scale above 1 strengthens alignment with the conditioning input but may introduce artifacts or reduce sample diversity. Therefore, the choice of guidance scale requires empirical testing and may differ depending on a specific model’s behavior and method of conditioning.

5.2.6 Latent Diffusion Models

Latent diffusion models (LDMs) [24] learn the same noise-prediction objective as traditional diffusion models, but perform the diffusion process within a compressed

latent space to improve efficiency. Typically, LDMs use a pre-trained variational autoencoder (VAE) [25] to encode images $x \in \mathbb{R}^{H \times W \times C}$ into a compressed latent representation $z \in \mathbb{R}^{H' \times W' \times C'}$, where $H' < H$ and $W' < W$. A diffusion model (often a U-Net as in pixel-space diffusion models) then learns to predict noise in this latent space, with samples generated by decoding denoised latents to the original image resolution using the VAE decoder.

LDMs often incorporate conditioning mechanisms for text prompts or images using cross-attention with text embeddings or auxiliary conditioning networks such as ControlNet [26]. The improved efficiency of LDMs enables them to be trained on much larger, more diverse datasets than pixel-level diffusion models. As a result, large conditional LDMs such as Stable Diffusion [24] are frequently used as pre-trained backbones for a variety of fine-tuning tasks. However, because LDMs operate on compressed latent representations, they may fail to preserve fine-grained image details, leading to the loss of task-relevant information during image-to-image translation.

5.3 Methods

5.3.1 Overview

This section describes a diffusion-based image-to-image translation approach that operates directly in pixel space, with the goal of transferring low-level visual characteristics for sim-to-real adaptation. Starting from a partially corrupted simulated image, the model performs translation through an iterative denoising process. At each step, denoising is conditioned on a learned real-domain embedding, which guides the output toward the real domain, and on the original source image, which provides a stable reference to preserve structure.

The proposed method is a pixel-level adaptation of CycleNet [15], which performs unpaired image-to-image translation using latent diffusion models and prompt-based domain conditioning. The pixel-level approach makes two key modifications to Cy-

cleNet: (1) it operates directly in pixel space rather than in a compressed latent space, and (2) it conditions the model on learned domain embeddings rather than text prompt embeddings.

5.3.2 CycleNet

The original CycleNet architecture uses a pre-trained LDM backbone with an auxiliary conditioning network for image conditioning. Specifically, CycleNet builds on a Stable Diffusion [24] backbone and incorporates a ControlNet [26] conditioning branch, which processes the input image and injects spatial features into the backbone at multiple resolutions. During training, the encoder of the backbone is frozen, while the decoder is trainable. The ControlNet conditioning branch is a trainable copy of the backbone encoder that feeds spatial features into the backbone decoder using zero convolutions.

CycleNet takes four inputs in each forward pass: a noisy latent z_t , an image condition c_{img} , a conditional prompt embedding c^+ , and an unconditional prompt embedding c^- . CycleNet constructs prompt embeddings using CLIP, encoding text prompts representing the source and target domains (e.g., $c^- = \text{CLIP}(\text{summer})$ and $c^+ = \text{CLIP}(\text{winter})$). c_{img} and c^- are fed to the ControlNet, with c_{img} serving as the structural reference image and c^- representing its domain. z_t and c^+ are fed to the backbone, with c^+ informing the backbone which domain to denoise z_t toward. For instance, when translating a source image $x_0 \in \mathcal{X}$ to a target domain $\mathcal{Y}$, the ControlNet receives the source image as the image condition $c_{\text{img}} = x_0$ and the source prompt embedding $c^- = \text{CLIP}(\mathcal{X}_{\text{prompt}})$, while the backbone receives a noised latent of the source image z_t and the target prompt embedding $c^+ = \text{CLIP}(\mathcal{Y}_{\text{prompt}})$.

5.3.3 Pixel-Space CycleNet

The original CycleNet architecture performs the diffusion process in a compressed latent space, potentially losing salient low-level details for training downstream models. The proposed pixel-level implementation operates in pixel space, allowing the denoising network to directly learn and transfer the pixel-level characteristics of real data.

U-Net Backbone

The pixel-space CycleNet architecture adopts a standard U-Net as the diffusion model backbone in place of a pre-trained LDM. The U-Net encoder consists of four layers, halving image resolution and increasing channel width at each. The final encoder layer is followed by a bottleneck layer, which performs additional processing without changing resolution or channel width. The U-Net decoder then mirrors the encoder layers, doubling image resolution and decreasing channel width at each while incorporating skip connections from the encoder.

Domain Conditioning

In place of CLIP prompt embeddings for source and target domains, the pixel-space implementation represents domain conditioning using learned embeddings $c_{\text{dom}} \in \mathbb{R}^k$. For a two-domain setting, this corresponds to a pair of fixed learnable vectors $c_{\mathcal{X}}, c_{\mathcal{Y}} \in \mathbb{R}^k$, which are randomly initialized and jointly optimized with the U-Net backbone. During pre-training, the U-Net backbone receives evenly split batches of source and target images paired with their corresponding domain embeddings. Each iteration, the domain embeddings are updated through gradient descent, encouraging the backbone to learn encodings of domain-specific characteristics within the dataset. By learning domain embeddings during pre-training, the embeddings are able capture unique features of the dataset that may not be represented within CLIP prompt

embeddings.

Domain conditioning is injected into the U-Net backbone through both cross-attention at the lowest spatial resolution layers and through adaptive group normalization (AdaGN) modulation within all residual blocks of the encoder, bottleneck, and decoder. Given a layer’s normalized spatial feature map $x \in \mathbb{R}^{H \times W \times C}$ and a domain embedding $c_{\text{dom}} \in \mathbb{R}^k$, cross-attention is applied such that each spatial position of the feature map queries a single domain token. In practice, the spatial feature map is flattened into spatial query tokens $Q \in \mathbb{R}^{HW \times C}$, while the domain embedding is projected to a single context token providing keys and values $K, V \in \mathbb{R}^{1 \times C}$. For multi-head attention with h heads, each head splits channels into dimension $d_k = C/h$, and the attention output is computed as:

$$\begin{aligned} \text{Attn}(Q, K, V) &= \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \\ Q &\in \mathbb{R}^{HW \times d_k}, \quad K, V \in \mathbb{R}^{1 \times d_k}. \end{aligned}$$

AdaGN applies a learned scale and shift to the normalized feature map within each residual block. Given a domain embedding $c_{\text{dom}} \in \mathbb{R}^k$, a small MLP within each residual block learns to produce a domain-dependent scale and shift $\gamma, \beta \in \mathbb{R}^C$ to apply to the block’s group-normalized feature map $x \in \mathbb{R}^{H \times W \times C}$:

$$\begin{aligned} \gamma, \beta &= \text{MLP}(c_{\text{dom}}) \\ \text{AdaGN}(x, c_{\text{dom}}) &= \gamma \cdot x + \beta. \end{aligned}$$

Because the cross-attention mechanism only applies to the lowest spatial resolutions within the U-Net and treats the domain embedding as a single token, it essentially acts as a global domain-dependent bias. Therefore, AdaGN is the primary driver behind low-level, spatially dependent adjustments for domain embeddings.

Spatial Conditioning

Spatial conditioning is incorporated using a ControlNet side network, initialized as a trainable copy of the pre-trained U-Net encoder. The ControlNet stores features

at each encoder layer, injecting them into the backbone decoder via zero convolutions. Zero convolutions are 1×1 convolutional layers that are initialized with zero weights and biases, effectively disabling the ControlNet at the beginning of training and allowing its contribution to gradually grow.

5.3.4 Training Objectives

The pixel-space approach follows the same training objectives implementation as the official CycleNet repository. The training objectives include a standard diffusion reconstruction loss, a CycleGAN-like cycle-consistency loss, and an invariance loss. Computing the objectives requires multiple forward passes under different domain-conditioning and image-conditioning inputs.

It is important to note that the two domain embeddings do not purely represent the translation direction, but rather their roles within the architecture. The domain embedding fed to the ControlNet specifies the conditioning image’s domain of origin, while the domain embedding fed to the backbone specifies the destination domain of the denoised output. We denote the domain-conditioning pair as $c_{A \rightarrow B}$, where A is the domain embedding fed to the ControlNet and B is the domain embedding fed to the backbone (e.g., $c_{\mathcal{X} \rightarrow \mathcal{Y}}$ means: interpret the conditioning image as originating from domain $\mathcal{X}$ and denoise toward domain $\mathcal{Y}$).

In addition, we follow the standard diffusion process introduced in 5.2.1, using a variance schedule $\{\beta_t\}_{t=1}^T$ with corresponding coefficients $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. The model’s noise predictions are denoted as $\epsilon_\theta(x_t, t, c_{A \rightarrow B}, c_{\text{img}})$, where x_t is a noised image at timestep t , $c_{A \rightarrow B}$ is the domain-conditioning pair, and c_{img} is the conditioning image. For brevity, we introduce the shorthand

$$\epsilon_\theta^{A \rightarrow B}(x_t, c_{\text{img}}) := \epsilon_\theta(x_t, t, c_{A \rightarrow B}, c_{\text{img}}).$$

Reconstruction Loss

Given a clean source image x_0 and timestep t , we sample Gaussian noise $\epsilon_x \sim \mathcal{N}(0, I)$ and obtain the noised image x_t using Eq. (5.1). We then perform a forward pass to obtain the noise prediction $\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{X}}(x_t, x_0)$:

$$\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{X}}(x_t, x_0) = \epsilon_\theta(x_t, t, c_{\mathcal{X} \rightarrow \mathcal{X}}, x_0).$$

This initial forward pass simulates a standard source-domain noise prediction, with the model attempting to denoise x_t back into the source domain $\mathcal{X}$ while given the clean image x_0 as the conditioning image. This establishes the source-domain as a fixed point, meaning the network retains its pre-trained ability to simply reconstruct source images and preserve their content. Reconstruction loss $\mathcal{L}_{\text{recon}}$ is then defined as

$$\mathcal{L}_{\text{recon}} = \mathbb{E}_{x_0, t, \epsilon_x} \left\| \epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{X}}(x_t, x_0) - \epsilon_x \right\|_2^2.$$

Cycle-Consistency Loss

The CycleNet paper [15] presents a single cycle-consistency loss term, but the official implementation splits the term into two complementary losses: a diffusion analogue of standard cycle-consistency loss $\mathcal{L}_{\text{cycle}}$, which encourages reversibility through the generated target-domain intermediate, and a source-anchored consistency loss $\mathcal{L}_{\text{consis}}$, which encourages simple reversibility by preserving source content.

Given the noised source image x_t , we perform a forward pass to obtain the noise prediction $\hat{\epsilon}^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0)$, denoising x_t toward domain $\mathcal{Y}$ while using the clean source image x_0 as the conditioning image:

$$\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0) = \epsilon_\theta(x_t, t, c_{\mathcal{X} \rightarrow \mathcal{Y}}, x_0).$$

This forward pass performs the same translation-denoising step used at inference time when translating images from $\mathcal{X}$ to $\mathcal{Y}$. From this noise prediction, we obtain an

estimate of the clean $\mathcal{Y}$ -domain image $\hat{y}_0$ using Eq. (5.2):

$$\hat{y}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0)}{\sqrt{\bar{\alpha}_t}}.$$

As an important implementation detail, gradients to the U-Net backbone are disabled during this forward pass. This allows the clean estimate $\hat{y}_0$ to serve as a pseudo $\mathcal{Y}$ -domain sample for the subsequent consistency and invariance objectives, preventing the backbone from directly adapting through its own generated sample. In addition, when $\hat{y}_0$ is reused as the conditioning image in later forward passes, it is detached from the computation graph. This treats $\hat{y}_0$ as a fixed conditioning image, preventing the model from optimizing the condition that it also produced. Detaching gradients through generated $\mathcal{Y}$ -domain samples to the backbone helps preserve the generative prior of the pre-trained U-Net while allowing the ControlNet to learn structure preservation and translation control. In our experiments, this gradient-detachment strategy significantly improved training stability.

We noise the $\mathcal{Y}$ -domain estimate $\hat{y}_0$ to y_t by sampling Gaussian noise $\epsilon_y \sim \mathcal{N}(0, I)$ and applying Eq. (5.1). We then perform a forward pass using domain conditioning $c_{\mathcal{Y} \rightarrow \mathcal{X}}$ and $\text{sg}[\hat{y}_0]$ as the conditioning image, where $\text{sg}[\cdot]$ denotes the stop-gradient operation:

$$\epsilon_\theta^{\mathcal{Y} \rightarrow \mathcal{X}}(y_t, \text{sg}[\hat{y}_0]) = \epsilon_\theta(y_t, t, c_{\mathcal{Y} \rightarrow \mathcal{X}}, \text{sg}[\hat{y}_0]).$$

In both $\mathcal{L}_{\text{cycle}}$ and $\mathcal{L}_{\text{consis}}$, the forward noise prediction $\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0)$ is detached when computing the MSE loss, treating the forward translation as a fixed estimate. This forces the reverse branches to estimate the remaining noise needed to complete the cycle instead of allowing the forward prediction to learn to directly absorb the error. Cycle loss $\mathcal{L}_{\text{cycle}}$ is then defined as

$$\mathcal{L}_{\text{cycle}} = \mathbb{E}_{x_0, t, \epsilon_x, \epsilon_y} \left\| \text{sg}[\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0)] + \epsilon_\theta^{\mathcal{Y} \rightarrow \mathcal{X}}(y_t, \text{sg}[\hat{y}_0]) - \epsilon_x - \epsilon_y \right\|_2^2.$$

The source-anchored consistency loss uses the same pseudo $\mathcal{Y}$ -domain estimate $\hat{y}_0$,

but instead of performing a true $\mathcal{Y} \rightarrow \mathcal{X}$ reverse translation, it attempts to recover the source image x_0 under full source-domain conditioning. Unlike cycle loss, consistency loss does not optimize the full translation cycle $\mathcal{X} \rightarrow \mathcal{Y} \rightarrow \mathcal{X}$ and treats $\hat{y}_0$ as fixed by detaching it. This prevents the consistency loss from affecting the earlier $\mathcal{X} \rightarrow \mathcal{Y}$ translation and keeps it focused on training the source-anchored recovery path. Specifically, the model predicts $\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{X}}(\text{sg}[y_t], x_0)$ using $\mathcal{X}$ -domain conditioning on both sides of the network and using x_0 as the conditioning image:

$$\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{X}}(\text{sg}[y_t], x_0) = \epsilon_\theta(\text{sg}[y_t], t, c_{\mathcal{X} \rightarrow \mathcal{X}}, x_0).$$

Consistency loss enforces that following an $\mathcal{X} \rightarrow \mathcal{Y}$ translation, the generated pseudo $\mathcal{Y}$ -domain sample should retain enough source information that the model can recover x_0 when fully anchored by $\mathcal{X}$ -domain conditioning. Consistency loss $\mathcal{L}_{\text{cons}}$ is defined as

$$\mathcal{L}_{\text{cons}} = \mathbb{E}_{x_0, t, \epsilon_x, \epsilon_y} \left\| \text{sg}[\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0)] + \epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{X}}(\text{sg}[y_t], x_0) - \epsilon_x - \epsilon_y \right\|_2^2.$$

Invariance Loss

The invariance loss term encourages the model to perform $\mathcal{X} \rightarrow \mathcal{Y}$ translation so that the resulting sample $\hat{y}_0$ is invariant to further target-domain translation, i.e., the model should perceive $\hat{y}_0$ as a valid sample from $\mathcal{Y}$. The additional $\mathcal{Y} \rightarrow \mathcal{Y}$ translation is performed by denoising x_t toward the $\mathcal{Y}$ -domain while using $\hat{y}_0$ as the conditioning image:

$$\epsilon_\theta^{\mathcal{Y} \rightarrow \mathcal{Y}}(x_t, \text{sg}[\hat{y}_0]) = \epsilon_\theta(x_t, t, c_{\mathcal{Y} \rightarrow \mathcal{Y}}, \text{sg}[\hat{y}_0]).$$

Invariance loss is then defined as

$$\mathcal{L}_{\text{invar}} = \mathbb{E}_{x_0, t, \epsilon_x, \epsilon_y} \left\| \epsilon_\theta^{\mathcal{Y} \rightarrow \mathcal{Y}}(x_t, \text{sg}[\hat{y}_0]) - \text{sg}[\epsilon_\theta^{\mathcal{X} \rightarrow \mathcal{Y}}(x_t, x_0)] \right\|_2^2.$$

Full Training Objective

For clarity, the losses are written for the $\mathcal{X} \rightarrow \mathcal{Y}$ translation direction. In practice, the training objectives are computed symmetrically for both $\mathcal{X} \rightarrow \mathcal{Y}$ and $\mathcal{Y} \rightarrow \mathcal{X}$ directions. The CycleNet training objective for the $\mathcal{X} \rightarrow \mathcal{Y}$ translation direction is computed as the weighted sum of the individual loss terms:

$$\mathcal{L}_{\mathcal{X} \rightarrow \mathcal{Y}} = \lambda_{\text{recon}} \mathcal{L}_{\text{recon}} + \lambda_{\text{cycle}} \mathcal{L}_{\text{cycle}} + \lambda_{\text{consis}} \mathcal{L}_{\text{consis}} + \lambda_{\text{invar}} \mathcal{L}_{\text{invar}}.$$

The full CycleNet training objective for both translation directions is therefore

$$\mathcal{L}_{\text{CycleNet}} = \mathcal{L}_{\mathcal{X} \rightarrow \mathcal{Y}} + \mathcal{L}_{\mathcal{Y} \rightarrow \mathcal{X}}.$$

5.3.5 Training Setup

Datasets

We collected a variety of RGB remote sensing datasets for our experiments with the aim of improving downstream land cover mapping segmentation models. We used the SynRS3D dataset [27] for all of our simulated data, using it in both diffusion model training and sim-to-real translation for downstream segmentation models.

Our real datasets consisted of remote sensing imagery from the IEEE Data Fusion Challenge (DFC) 2018 [28] and 2019 [29], GeoNRW [30], and OpenEarthMap (OEM) [31]. All the real datasets were used for diffusion model training, however, only the OEM dataset was used for land cover mapping training and evaluation, as it follows the same class labeling scheme as SynRS3D.

Images vary in size between datasets, typically ranging between 512×512 and 1024×1024 . Due to the computational constraints of pixel-space diffusion model training, we split all images into 256×256 tiles. We similarly split the segmentation masks in the OEM and SynRS3D datasets to preserve alignment. Table 5.1 details each dataset’s source location and sample count after tiling.

Table 5.1: Summary of real and simulated datasets used in this work.

Dataset	Domain	Region	# Images
DFC18	Real	Houston	341
DFC19 (JAX)	Real	Jacksonville, USA	16,240
DFC19 (OMA)	Real	Omaha, USA	28,288
GeoNRW	Real	North Rhine-Westphalia, Germany	490,497
OEM	Real	Global	39,598
SynRS3D	Sim	Synthetic (global)	557,272

Sampling Strategy

When training the U-Net backbone and CycleNet models, we balance each batch evenly between real and simulated samples, ensuring that the models are not biased towards one domain due to imbalanced random sampling. We do not perform any balancing based on specific datasets, location, or environment. As a result, there may be an imbalance in observed regions, city layouts, urban and rural environments, etc. However, because only the SynRS3D and OEM datasets contain segmentation masks, there was no efficient way to balance sampling based on image contents.

We performed two training experiments for the U-Net backbone and CycleNet models using different dataset splits: (1) all real and sim data and (2) only OEM real data and all sim data. These training dataset splits serve to answer two different questions: (1) do general learned real features transfer between datasets? and (2) do dataset-specific real features improve downstream performance more than general features? Table 5.2 describes the training datasets and real and simulated sample counts for both experiments.

Table 5.2: U-Net and CycleNet training dataset splits.

Training Split	Included Datasets					# Sim	# Real
	DFC18	DFC19	GeoNRW	OEM	SynRS3D		
All Real	✓	✓	✓	✓	✓	557,272	574,964
OEM Only				✓	✓	557,272	39,598

Hyperparameters

The training configuration for the U-Net backbone models is summarized in Table 5.3, and the configuration for the CycleNet models is summarized in Table 5.4. Note that the architecture and diffusion parameters are absent from the CycleNet table, as they are inherited from the U-Net. The all-real and oem-only models share the same architecture and optimization parameters for both the U-Net backbone and CycleNet models.

Table 5.3: U-Net backbone training configuration.

Parameter	Value
Architecture	
Base channels	128
Channel multipliers	[1, 2, 4, 4]
Residual blocks per level	2
Attention resolutions	32×32
Normalization	GroupNorm (32 groups)
Activation	SiLU
Time embedding dimension	512
Domain embedding dimension	$k = 128$
Diffusion	
Timesteps (T)	1000
Beta schedule	Linear
Beta range	$10^{-4} \rightarrow 0.02$
Training Parameters	
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	64
Training steps	400,000
EMA decay	0.9999
Image resolution	256×256

Table 5.4: CycleNet training configuration.

Parameter	Value
Training Parameters	
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	64
Training steps	50,000
EMA decay	0.9999
Image resolution	256×256
λ_{recon}	1.0
λ_{cycle}	0.01
λ_{consis}	0.01
λ_{invar}	0.1

5.3.6 Inference Procedure

During inference, image-to-image translation with CycleNet is controlled by three parameters: the classifier-free guidance (CFG) weight, the noise strength, and the number of DDIM sampling steps.

Classifier-Free Guidance Weight

The CFG weight w controls the coefficient applied in Eq. 5.3 to the denoising trajectory, pulling the output toward the target-domain noise prediction. In other words, increasing the CFG weight results in more target-like outputs. However, very large weights can push the trajectory out of the target domain, resulting in exaggerated target-domain artifacts and degradation in output quality. In our experiments, we swept over CFG weights of $w \in \{1, 2, 3, 4, 5\}$, as we found that increasing w above 5 resulted in low-quality samples.

Noise Strength

The noise strength γ controls how much noise is injected into the source image prior to translation. Specifically, the noise strength is defined as the proportion of the total timesteps T to noise to, where $\gamma \in [0, 1]$. Selecting a reliable noise strength for realistic sim-to-real translation requires a balance between preserving structural information from the source image and disturbing the image enough for the CycleNet model to make alterations. Although noise is injected using a linear noise schedule β_t in Eq. 5.1, the signal power of the source image x_0 at timestep t , given by $\bar{\alpha}_t x_0$, decays approximately exponentially as t increases. For instance, using our linear noise schedule from $\beta_1 = 0.0001$ to $\beta_T = 0.02$, the remaining signal power of the source image at timestep $t = 500$ is ≈ 0.08 . As shown in Fig. 5.1, the simulated images become very corrupted by $\gamma = 0.5$. Therefore, we chose to perform inference over noise strength values of $\gamma \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$, as values of γ above 0.5 destroyed too much information in the source images.

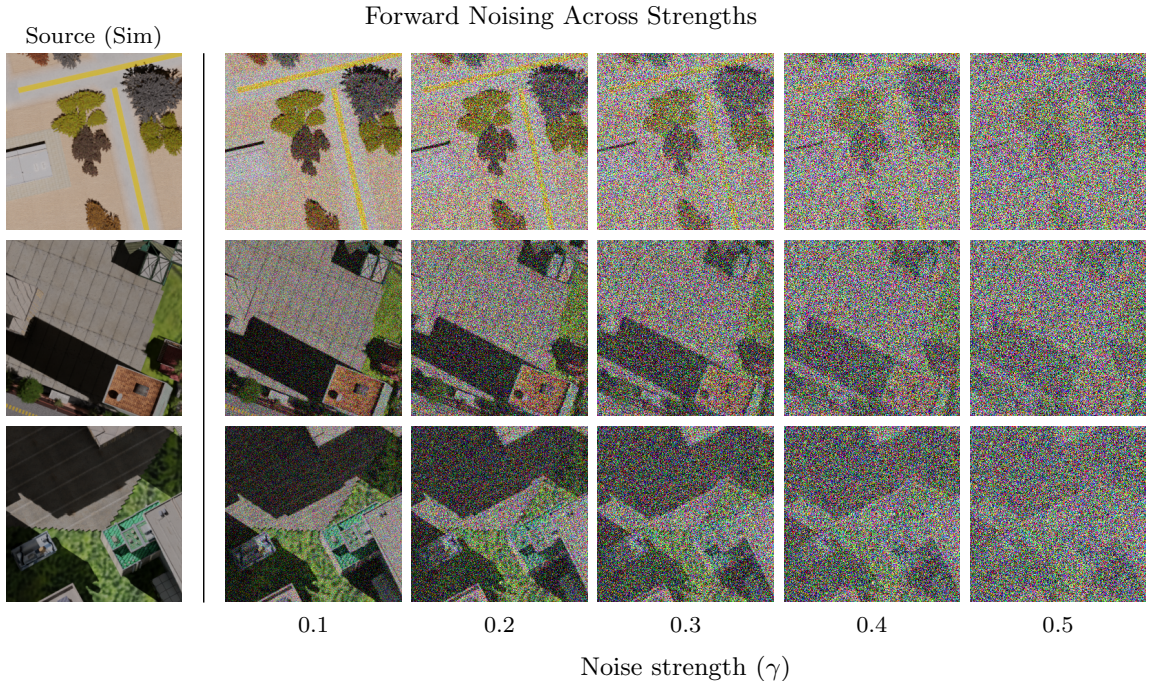


Figure 5.1: Forward noising of simulated images over noise strengths.

DDIM Sampling Steps

When denoising an image which has been noised to timestep $t = t_S$, DDIM sampling performs S denoising steps along a uniformly spaced subsequence of timesteps $\{t_s\}_{s=1}^S$, where $t_S > t_{S-1} > \dots > t_1 = 1$. For example, if an image has been noised to timestep $t_S = 100$ and $S = 3$, the denoising steps occur at $t_3 = 100$, $t_2 = 50$, and $t_1 = 1$. When applying a noise strength γ with S DDIM steps, we define the maximum noising DDIM step

$$s_\gamma = \lfloor \gamma S \rfloor,$$

then noise the image to t_{s_γ} . In other words, when using DDIM sampling, the noise strength defines which subsampled DDIM timestep $\{t_s\}_{s=1}^S$ to noise to. In our experiments, we set $S = 50$ as it improves efficiency with negligible degradation in sample quality compared to common higher numbers of steps, such as $S = 100$ and $S = 200$.

5.3.7 Evaluation Metrics

To measure both the realism of translated data and its effect on downstream tasks, we evaluate translated images using image-level, feature-based, and distribution-wide metrics.

Image-Level Metrics

To assess how well translated images preserve the perceptual and structural content of their source simulated images, we use Learned Perceptual Image Patch Similarity (LPIPS) [32]. LPIPS provides a measure of human perceptual similarity, comparing deep feature activations from a pre-trained network with weights learned from human visual judgments. It performs feature comparisons per spatial location across multiple layers, making it sensitive to differences in both structure and appearance. As LPIPS

performs comparisons between individual pairs of images, it is not directly suitable as a similarity measure between datasets. Therefore, in our experiments, we compute the average LPIPS between simulated images and their corresponding translated images to measure their general perceptual change.

Distribution-Level Metrics

We additionally evaluate translated data using distribution-level metrics, comparing translated datasets with their target-domain real-world datasets, measuring how well the translated data captures the underlying features of the real domain.

Fréchet inception distance (FID) [33] measures the distance between the distribution of a set of generated images and the distribution of a set of real images within the feature space of an InceptionV3 network [34]. Specifically, it compares features from the 2048-dimensional activations of the final pooling layer of the network. FID measures the distance between distributions by modeling them as multivariate Gaussians and comparing their mean and covariance matrices. For a real feature distribution and generated feature distribution with mean and covariance (μ_r, Σ_r) and (μ_g, Σ_g) , respectively, FID is defined as:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

However, computing the Fréchet distance between feature distributions is not restricted to InceptionV3 features. We additionally compute the Fréchet distance on the features of a DeepLabv3 segmentation model [35] trained on OEM and SynRS3D data, measuring how well translated images capture task-relevant features for downstream segmentation. We refer to this metric as *DeepLab-FID*.

Feature Projections

To provide a qualitative visualization of the relationship between real, simulated, and translated image distributions, we use Uniform Manifold Approximation and Projection (UMAP) [36] to project their feature embeddings into a shared two-dimensional space. UMAP is a graph-based dimensionality reduction technique that aims to preserve the local neighborhoods and structure of the high-dimensional data manifold within a low-dimensional space. Therefore, UMAP projections provide an approximate two-dimensional representation of how translated images fit relative to real and simulated images within feature space.

Downstream Metrics

Semantic segmentation performance is measured using mean Intersection over Union (mIoU), per-class IoU, and pixel accuracy. mIoU is defined as the average intersection-over-union between the ground truth segmentation masks and predicted masks across all classes. We additionally measure per-class IoU to identify how the inclusion of translated data affects segmentation performance for each semantic class label.

5.3.8 Model Selection

To efficiently explore the search space of CycleNet model checkpoints and sampling parameters, we employ a multi-objective selection procedure to identify promising configurations.

Candidate Configurations

The selection process begins with an initial limited set of model checkpoints, classifier-free guidance weights, and noise strengths for testing. Specifically, we evaluate model checkpoints at training iterations $m \in \{2,500, 10,000, 20,000, 30,000\}$, and perform

inference for all combinations of classifier-free guidance weights $w \in \{1, 2, 3, 4, 5\}$ and noise strengths $\gamma \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$.

Multi-Objective Selection

Prior to evaluation, we sample a fixed set of 2,000 simulated images for translation. For each unique combination (m, w, γ) , the simulated images are translated and saved for analysis, resulting in 2,000 translated samples for each configuration. Using a held-out target-domain dataset of 2,000 real images, we then compute FID, DeepLab-FID, and LPIPS for each translated dataset. To prioritize configurations that improve both generic and task-specific realism, we then reduce the candidates to the Pareto-optimal set of configurations with respect to FID and DeepLab-FID. We then select the configuration with the lowest LPIPS for each model checkpoint, discouraging structurally unstable configurations.

Final Evaluation Configurations

The final configurations selected for evaluation are detailed in Table 5.5. These configurations are used for all subsequent experiments.

Table 5.5: Final configurations selected for evaluation.

Checkpoint (m)	CFG Weight (w)	Noise Strength (γ)
20k	5	0.1
30k	5	0.2

5.4 Experiments

5.4.1 Overview

This section evaluates the effect of using pixel-space CycleNet-translated simulated imagery for downstream semantic segmentation. To measure both the visual real-

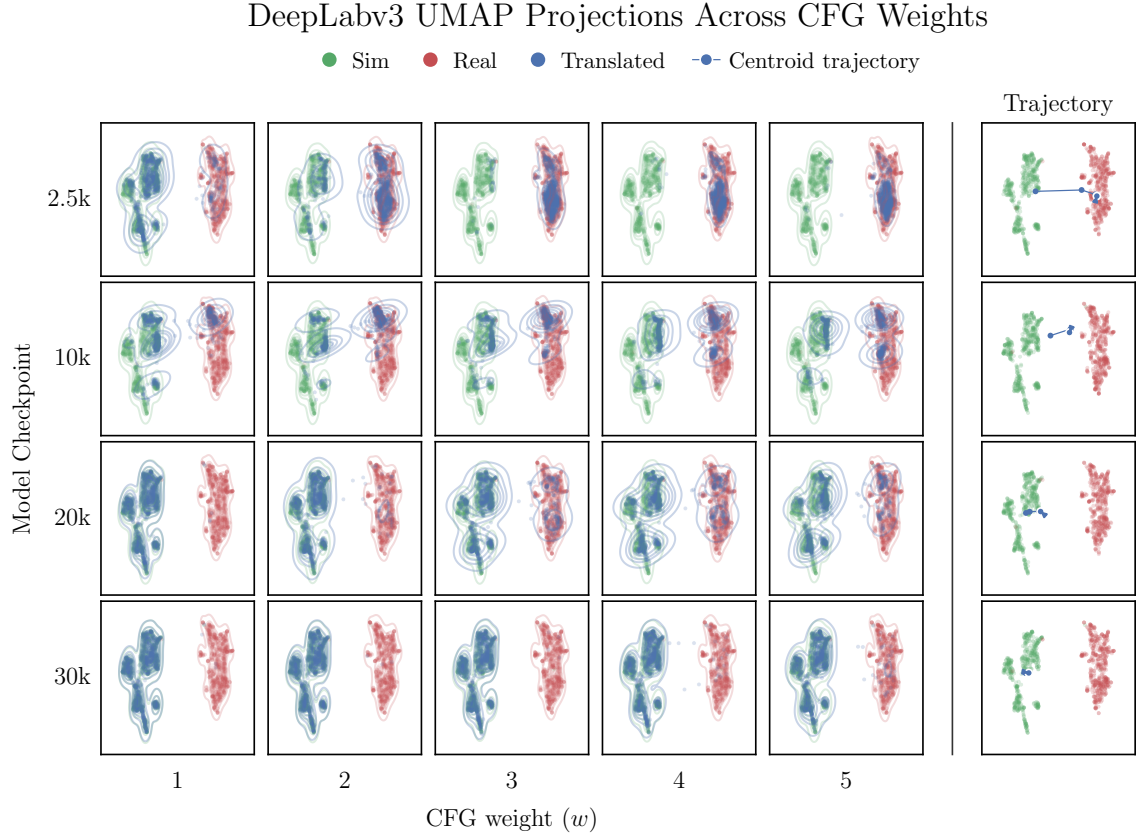


Figure 5.2: Caption

ism and downstream task-relevance of translated data, experiments are conducted to measure two key aspects: (1) the similarity between the translated and real image distributions, and (2) how the translated data impacts segmentation performance on real OEM imagery.

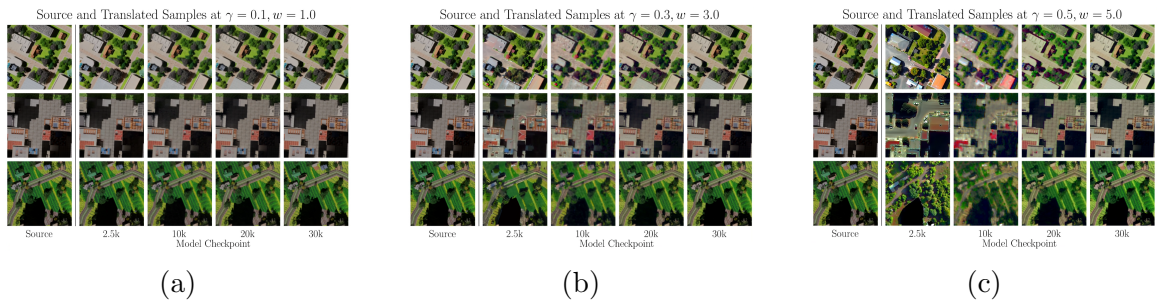


Figure 5.3: Caption

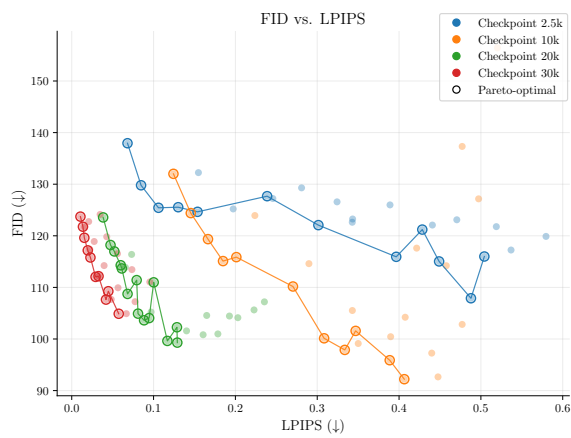


Figure 5.4: Enter Caption

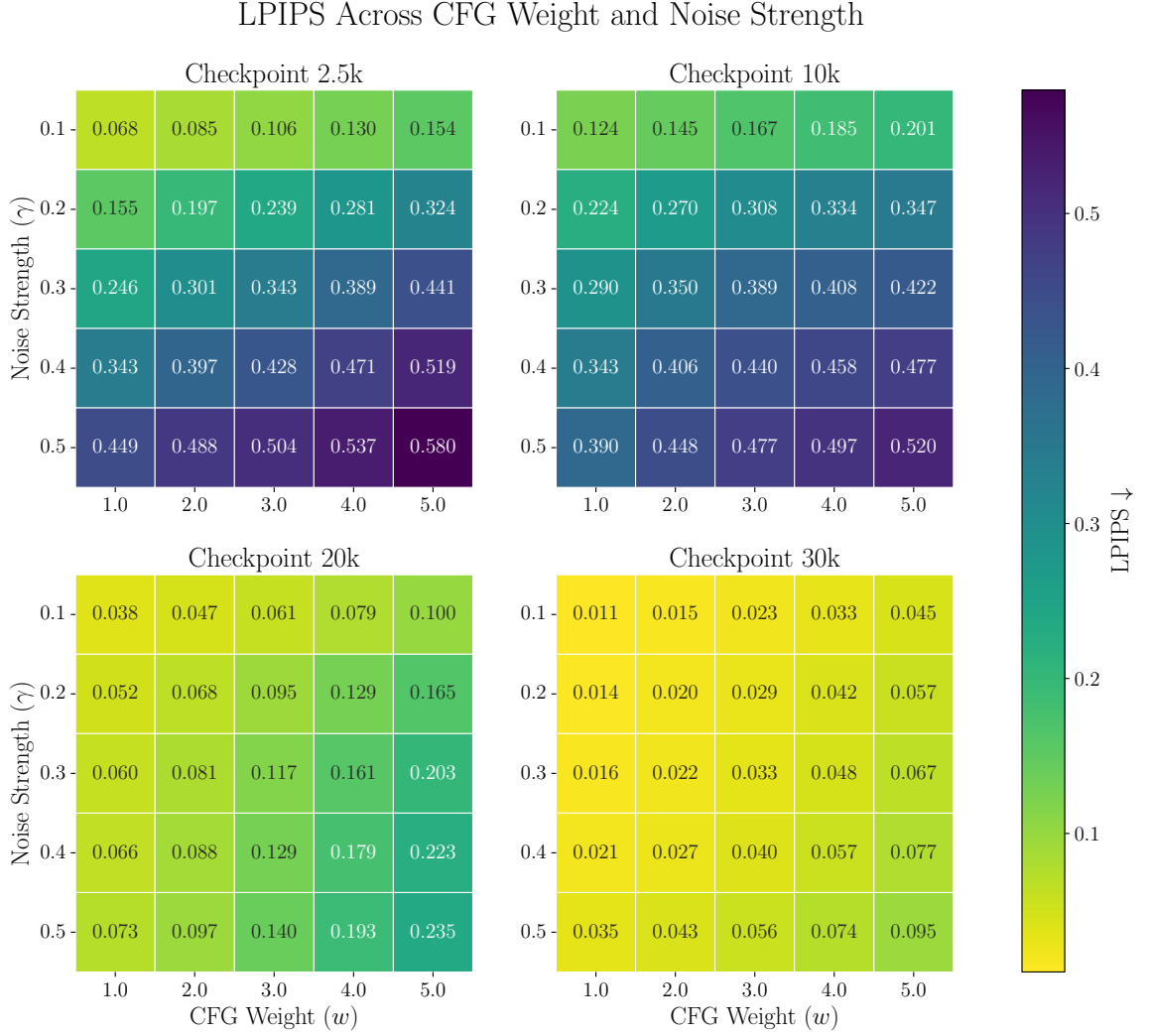


Figure 5.5: Mean LPIPS between translated images and their source simulated images across CFG weight and noise strength configurations.

5.4.2 Translation Behavior

Training Progression

5.4.3 Multi-Objective Selection Analysis

Metric Trade-offs

5.4.4 Downstream-Task Evaluation

Experimental Setup

Quantitative Results

Qualitative Results

Discussion and Failure Cases

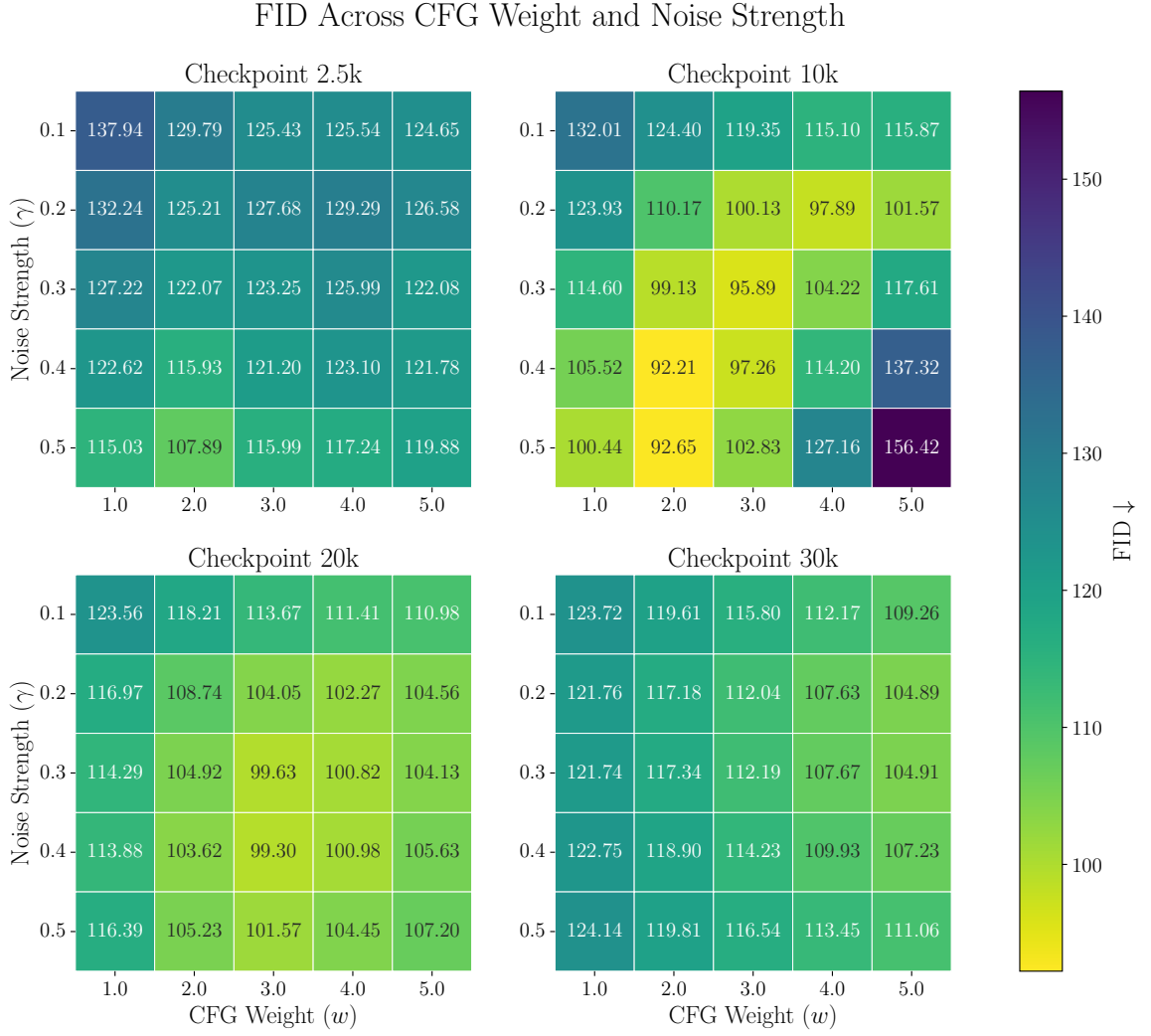


Figure 5.6: FID between translated images and their source simulated images across CFG weight and noise strength configurations.

6 Conclusion

Bibliography

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems* (F. Pereira, C. Burges, L. Bottou, and K. Weinberger, eds.), vol. 25, Curran Associates, Inc., 2012.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [3] Epic Games, “Unreal Engine.” Epic Games, Cary, NC, USA, 25 April 2019. Available online: <https://www.unrealengine.com>, 2019. (Accessed: 14 June 2023).
- [4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.
- [5] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. March, and V. Lempitsky, “Domain-adversarial training of neural networks,” *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.
- [6] B. Sun, J. Feng, and K. Saenko, “Correlation alignment for unsupervised domain adaptation,” *CoRR*, vol. abs/1612.01939, 2016.

- [7] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, “Autoaugment: Learning augmentation policies from data,” *CoRR*, vol. abs/1805.09501, 2018.
- [8] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “Randaugment: Practical data augmentation with no separate search,” *CoRR*, vol. abs/1909.13719, 2019.
- [9] P. Isola, J. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” *CoRR*, vol. abs/1611.07004, 2016.
- [10] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” in *Computer Vision (ICCV), 2017 IEEE International Conference on*, 2017.
- [11] C. Meng, Y. Song, J. Song, J. Wu, J. Zhu, and S. Ermon, “Sdedit: Image synthesis and editing with stochastic differential equations,” *CoRR*, vol. abs/2108.01073, 2021.
- [12] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or, “Prompt-to-prompt image editing with cross attention control,” 2022.
- [13] C. Saharia, W. Chan, H. Chang, C. Lee, J. Ho, T. Salimans, D. Fleet, and M. Norouzi, “Palette: Image-to-image diffusion models,” SIGGRAPH ’22, (New York, NY, USA), Association for Computing Machinery, 2022.
- [14] C. H. Wu and F. De La Torre, “A latent space of stochastic diffusion models for zero-shot image editing and guidance,” in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 7344–7353, 2023.
- [15] S. Xu, Z. Ma, Y. Huang, H. Lee, and J. Chai, “Cyclenet: Rethinking cycle consistency in text-guided diffusion for image manipulation,” in *Advances in Neural Information Processing Systems* (A. Oh, T. Naumann, A. Globerson,

- K. Saenko, M. Hardt, and S. Levine, eds.), vol. 36, pp. 10359–10384, Curran Associates, Inc., 2023.
- [16] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sasstry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” 2021.
 - [17] P. Lei, “Fusing color imagery with unstable ai-derived depth estimates for object detection,” 2025. Manuscript submitted for review to SPIE Defense + Commercial Sensing 2025.
 - [18] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
 - [19] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” 2019.
 - [20] C. Veal, M. Lindsay, S. Kovaleski, D. Anderson, and S. Price, “Neural metric learning using contrastive anchors to enforce feature separability,” 12 2021.
 - [21] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” *CoRR*, vol. abs/1505.04597, 2015.
 - [22] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems* (H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, eds.), vol. 33, pp. 6840–6851, Curran Associates, Inc., 2020.
 - [23] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” 2022.
 - [24] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” *CoRR*, vol. abs/2112.10752, 2021.

- [25] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” 2022.
- [26] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” 2023.
- [27] J. Song, H. Chen, W. Xuan, J. Xia, and N. Yokoya, “Synrs3d: A synthetic dataset for global 3d semantic understanding from monocular remote sensing imagery,” *arXiv preprint arXiv:2406.18151*, 2024.
- [28] Y. Xu, B. Du, L. Zhang, D. Cerra, M. Pato, *et al.*, “Advanced multi-sensor optical remote sensing for urban land use and land cover classification: Outcome of the 2018 ieee grss data fusion contest,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 6, pp. 1709–1724, 2019. hal-02875492.
- [29] Y. Lian, T. Feng, J. Zhou, M. Jia, A. Li, Z. Wu, L. Jiao, M. Brown, G. Hager, N. Yokoya, R. Hänsch, and B. L. Saux, “Large-scale semantic 3-d reconstruction: Outcome of the 2019 ieee grss data fusion contest—part b,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 14, pp. 1158–1170, 2021.
- [30] G. Baier, A. Deschemps, M. Schmitt, and N. Yokoya, “Geonrw,” 2020.
- [31] J. Xia, N. Yokoya, B. Adriano, and C. Broni-Bediako, “Openearthmap: A benchmark dataset for global high-resolution land cover mapping,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 6254–6264, January 2023.
- [32] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” *CoRR*, vol. abs/1801.03924, 2018.

- [33] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, G. Klambauer, and S. Hochreiter, “Gans trained by a two time-scale update rule converge to a nash equilibrium,” *CoRR*, vol. abs/1706.08500, 2017.
- [34] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” *CoRR*, vol. abs/1512.00567, 2015.
- [35] L. Chen, G. Papandreou, F. Schroff, and H. Adam, “Rethinking atrous convolution for semantic image segmentation,” *CoRR*, vol. abs/1706.05587, 2017.
- [36] L. McInnes, J. Healy, and J. Melville, “Umap: Uniform manifold approximation and projection for dimension reduction,” 2020.